

IBM Business Automation Manager Open Editions Demos and Labs 2026

Preparing your IBM BAMOE Business Service Project for the runtime environment

V 1.0 (for IBM BAMOE 9.5.2)

Raul Mariano
raul.mariano@ibm.com

NOTICES

This information was developed for products and services offered in the USA.

IBM may not offer the products, services, or features discussed in this document in other countries. Consult your local IBM representative for information on the products and services currently available in your area. Any reference to an IBM product, program, or service is not intended to state or imply that only that IBM product, program, or service may be used. Any functionally equivalent product, program, or service that does not infringe any IBM intellectual property right may be used instead. However, it is the user's responsibility to evaluate and verify the operation of any non-IBM product, program, or service.

IBM may have patents or pending patent applications covering subject matter described in this document. The furnishing of this document does not grant you any license to these patents. You can send license inquiries, in writing, to:

IBM Director of Licensing
IBM Corporation
North Castle Drive, MD-NC119
Armonk, NY 10504-1785
United States of America

The following paragraph does not apply to the United Kingdom or any other country where such provisions are inconsistent with local law: INTERNATIONAL BUSINESS MACHINES CORPORATION PROVIDES THIS PUBLICATION "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. Some states do not allow disclaimer of express or implied warranties in certain transactions, therefore, this statement may not apply to you.

This information could include technical inaccuracies or typographical errors. Changes are periodically made to the information herein; these changes will be incorporated in new editions of the publication. IBM may make improvements and/or changes in the product(s) and/or the program(s) described in this publication at any time without notice.

Any references in this information to non-IBM websites are provided for convenience only and do not in any manner serve as an endorsement of those websites. The materials at those websites are not part of the materials for this IBM product and use of those websites is at your own risk.

IBM may use or distribute any of the information you supply in any way it believes appropriate without incurring any obligation to you.

Information concerning non-IBM products was obtained from the suppliers of those products, their published announcements or other publicly available sources. IBM has not tested those products and cannot confirm the accuracy of performance, compatibility or any other claims related to non-IBM products. Questions on the capabilities of non-IBM products should be addressed to the suppliers of those products.

This information contains examples of data and reports used in daily business operations. To illustrate them as completely as possible, the examples include the names of individuals, companies, brands, and products. All of these names are fictitious and any similarity to the names and addresses used by an actual business enterprise is entirely coincidental.

TRADEMARKS

IBM, the IBM logo, and ibm.com are trademarks or registered trademarks of International Business Machines Corp., registered in many jurisdictions worldwide. Other product and service names might be trademarks of IBM or other companies. A current list of IBM trademarks is available on the web at "Copyright and trademark information" at www.ibm.com/legal/copytrade.shtml.

Adobe, the Adobe logo, PostScript, and the PostScript logo are either registered trademarks or trademarks of Adobe Systems Incorporated in the United States, and/or other countries.

Cell Broadband Engine is a trademark of Sony Computer Entertainment, Inc. in the United States, other countries, or both and is used under license therefrom.

Intel, Intel logo, Intel Inside, Intel Inside logo, Intel Centrino, Intel Centrino logo, Celeron, Intel Xeon, Intel SpeedStep, Itanium, and Pentium are trademarks or registered trademarks of Intel Corporation or its subsidiaries in the United States and other countries.

IT Infrastructure Library is a Registered Trade Mark of AXELOS Limited.

ITIL is a Registered Trade Mark of AXELOS Limited.

Java and all Java-based trademarks and logos are trademarks or registered trademarks of Oracle and/or its affiliates.

Linear Tape-Open, LTO, the LTO Logo, Ultrium, and the Ultrium logo are trademarks of HP, IBM Corp. and Quantum in the U.S. and other countries.

Linux is a registered trademark of Linus Torvalds in the United States, other countries, or both.

Microsoft, Windows, Windows NT, and the Windows logo are trademarks of Microsoft Corporation in the United States, other countries, or both.

UNIX is a registered trademark of The Open Group in the United States and other countries.

© Copyright International Business Machines Corporation 2024.

This document may not be reproduced in whole or in part without the prior written permission of IBM.

US Government Users Restricted Rights - Use, duplication or disclosure restricted by GSA ADP Schedule Contract with IBM Corp.

Table of Contents

1	Introduction.....	4
1.1	IBM Business Automation Manager Open Editions.....	4
2	Lab Setup Instructions	5
2.1	Objective	5
2.2	Access the environment.....	5
2.2.1	About the Environment	5
3	Exercises	7
3.1	The Business Service Project	7
3.2	Exercise 1: Configure data persistence with PostgreSQL	8
3.3	Exercise 2: Configure Grafana and Prometheus for monitoring and observability	14
3.4	Exercise 3: Configure Keycloak for security and user management.....	28
3.4.1	Step 01: Configuring the Keycloak application.....	28
3.4.2	Step 02: Configuring the BAMOE Business Service Project.....	35
4	Consult Documentation and Communities	42

1 Introduction

These hands-on lab exercises are designed to guide you through the essential aspects of process automation using BAMOE Development Tools for VS Code. Whether you're a developer or an architect, these labs will provide the skills needed to use BAMOE Canvas and VS Code to effectively create workflow automation and decision-making designs in modern, cloud-native business automation solutions.

Notes:

- *Includes **four** exercises. We recommend completing them sequentially.*
- **Duration:** *Approximately 1 hour.*
- **Target audience:** *Anyone who wants to learn how to use IBM Business Automation Manager Open Editions.*

1.1 IBM Business Automation Manager Open Editions

IBM Business Automation Manager Open Editions (IBM BAMOE) is a powerful open-source solution that serves as a foundation platform for tailoring long-lasting business automation solutions for the hybrid cloud.

With a developer-centric approach, this comprehensive and flexible platform makes it easy for teams to collaborate through Open Standards and efficient development tools suited for different personas.

Each automation solution can be shaped to perfectly address each scenario: business applications are flexible and can effortlessly integrate with external systems of your existing architecture.

Designed for the hybrid cloud, IBM Business Automation Manager Open Editions, accelerates the application modernization and cloud adoption journeys, as the lightweight design tools, business applications and other product components can be containerized and deployed with popular technologies such as Kubernetes and OpenShift.

For more information, see IBM documentation and other useful links:

- [IBM Business Automation Manager Open Editions Documentation](#)
- [Open Editions Community](#)

2 Lab Setup Instructions

2.1 Objective

See in practice how to use the BAMOE MCP Server to enable your agents to interact with your BAMOE applications. This feature is still in **Tech Preview**, but it is already possible to have a good experience connecting Agents to DMNs. Consult the documentation and follow upcoming updates to stay informed about the latest developments.

This lab was divided into 3 main exercises, see below:

- **Exercise 01** - Configure data persistence with PostgreSQL.
- **Exercise 02** - Configure Grafana and Prometheus for monitoring and observability.
- **Exercise 03** - Configure Keycloak for security and user management.



To complete this lab, we assume you have already had some contact with IBM BAMOE; therefore, some navigation or explanations that we consider "basic" will not be covered in detail.

2.2 Access the environment

You received this email with instructions on how to access the environment using your IBMid. Otherwise, click [here](#) to reserve it.

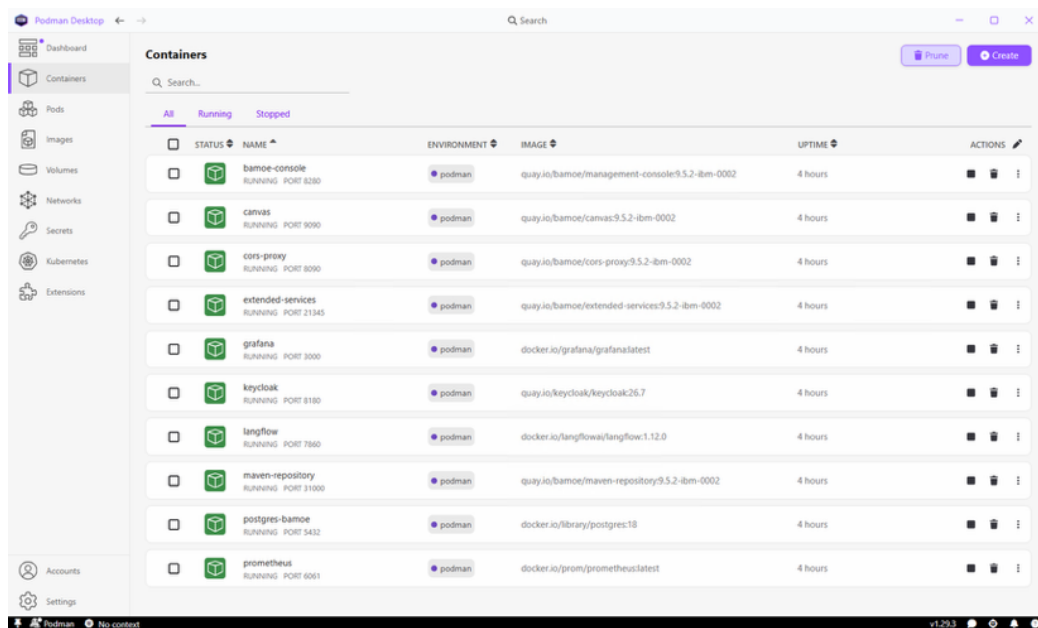
If necessary, this is the Windows credential:

```
User: techzone  
Password: IBMDem0s!
```

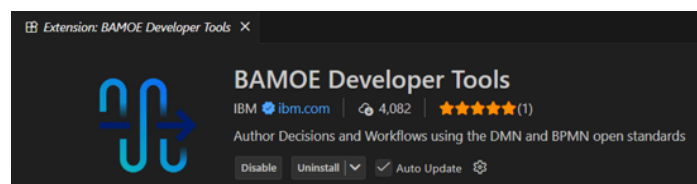
2.2.1 About the Environment

We built this environment so that you can practically use IBM Business Automation Manager Open Editions, so all applications are installed locally. To perform this lab, it will not be necessary to use any external services. Basically, we have this architecture:

- [Development](#) and [Runtime](#) applications are running in containers on Podman.



- For deployment and testing of your applications, VS Code with the [BAMOE Developer Tools](#) extension will be used.



Next, the first exercise begins. Happy studying!

3 Exercises

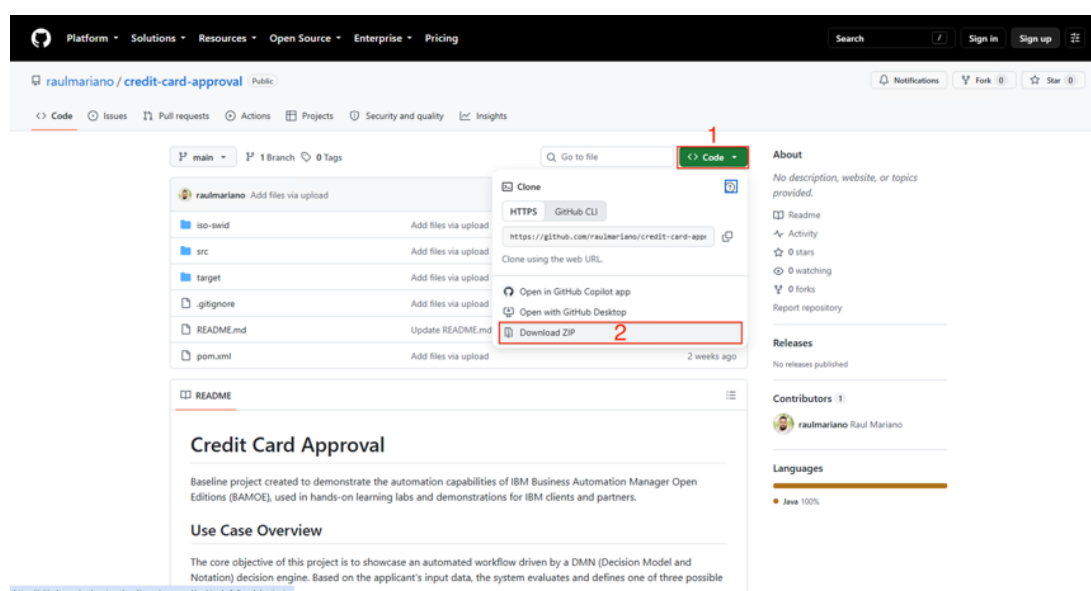
3.1 The Business Service Project

Go to GitHub to download the project we will use in the lab. You have likely already seen the Credit Card Approval use case; essentially, it features a main workflow that uses a DMN to determine three possible paths, one of which involves human approval. Therefore, please follow these instructions:

- The first step will be to download the project from GitHub. You can access the link and download the **.zip** file.

<https://github.com/raulmariano/credit-card-approval>

- Click on **"Code"** then on **"Download ZIP"**



- Save the **.zip** file and extract everything to a directory of your choice.
- Let's start by opening the project you downloaded in VS Code, all the configuration for the Business Service Project will be done there. You will notice that these two files are frequently cited:

File	Description
<code>your_project\pom.xml</code>	Responsible for managing dependencies, third-party libraries, and the project's build process.
<code>your_project\src\main\resources\application.properties</code>	The configuration file for the framework (Spring/Quarkus), where you define network ports, database connections, and runtime properties.

In short, `pom.xml` structures the tools the application needs to be built, while `application.properties` dictates how it should behave when running.

3.2 Exercise 1: Configure data persistence with PostgreSQL

In this first exercise, we will spin up PostgreSQL in Podman and configure it in the BAMOE Project Business Service.

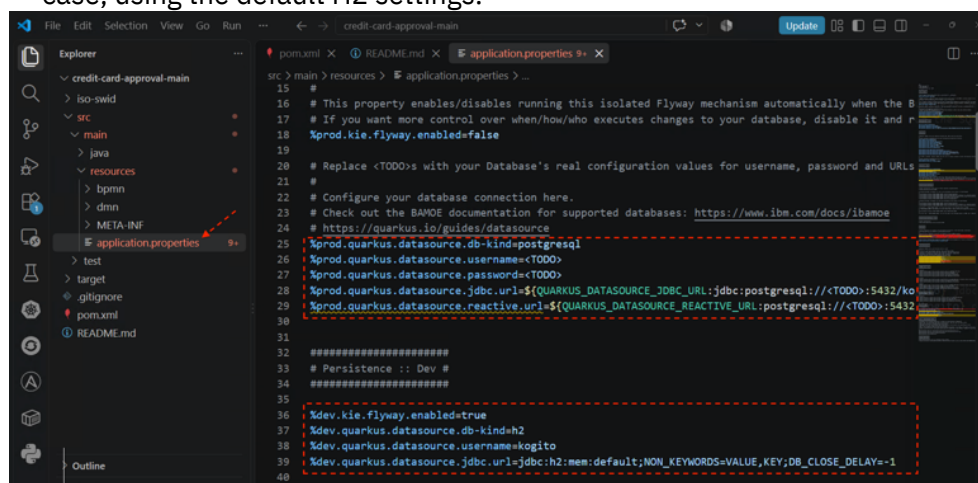
Note: Access the [Detailed System Requirements](#) for more information on other supported databases.

- When you use the standard IBM BAMOE accelerator, in this project case I use the "Workflows

(Quarkus + Maven)" , the project comes pre-configured with a default, temporary H2 database intended for use during development.

- The H2 database dependency is isolated within the "development" profile, so there is no longer a need to remove it from the pom file. You simply need to avoid invoking it in the Maven command: **"*mvn clean quarkus:dev -Pdevelopment*"**.
- Let's configure **application.properties**. In the first few lines of the file, you will find two database configuration blocks:

- One starting with "%prod", which is used for packaging and deploying to production (i.e., the production database settings).
- Second block starting with "%dev", which points to the development database, in this case, using the default H2 settings.



- Open the terminal to deploy the PostgreSQL container in Podman. Run this command:

```
podman run -d --name postgresql -e POSTGRES_USER=bamoe-adm -e
POSTGRES_PASSWORD=bamoe -e POSTGRES_DB=bamoe-db -p 127.0.0.1:5433:5432
postgres:18
```

To deploy and configure another database, consult the [IBM documentation](#).

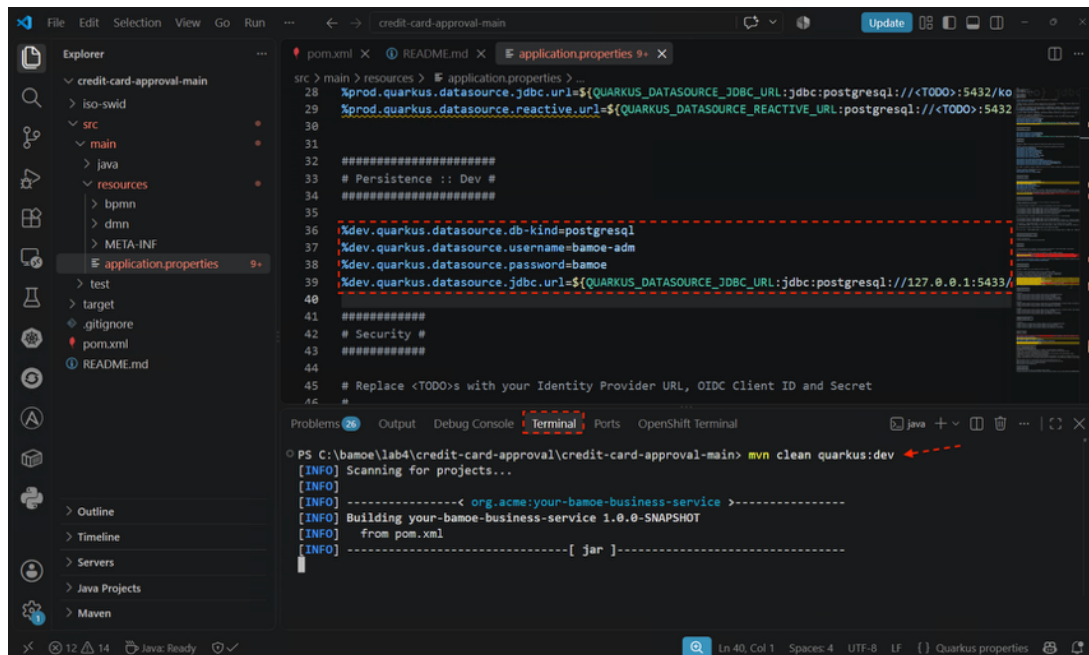
- In this exercise, we will work only in Development mode, so we will replace the H2 database configurations. Copy these PostgreSQL database configurations and paste them into your **"application.properties"** file:

```
%dev.quarkus.datasource.db-kind=postgresql
%dev.quarkus.datasource.username=bamoe-adm
```

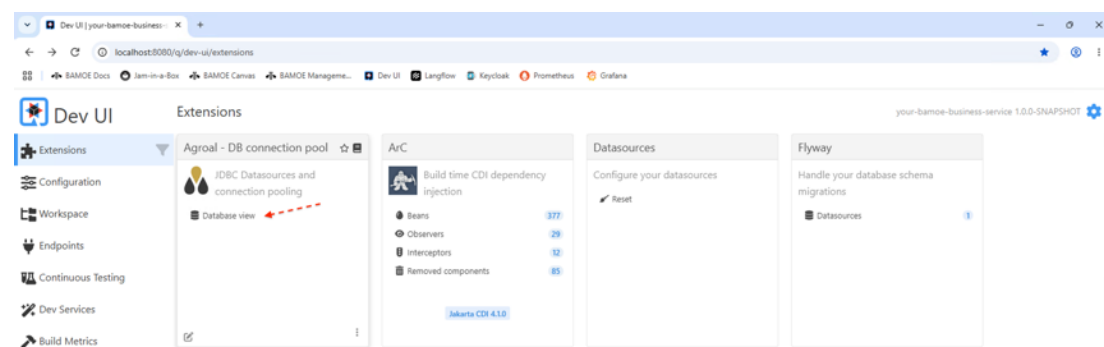
```
%dev.quarkus.datasource.password=bamoe
%dev.quarkus.datasource.jdbc.url=${QUARKUS_DATASOURCE_JDBC_URL:jdbc:pos
gresql://127.0.0.1:5433/bamoe-db}
```

- Save the changes.
- Now open the terminal and run the command:

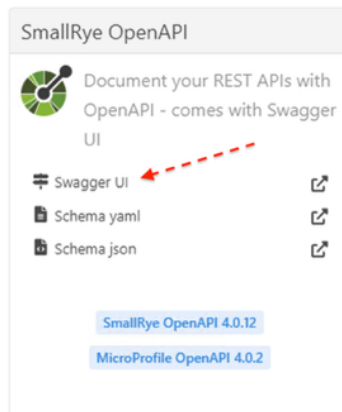
```
mvn clean quarkus:dev
```



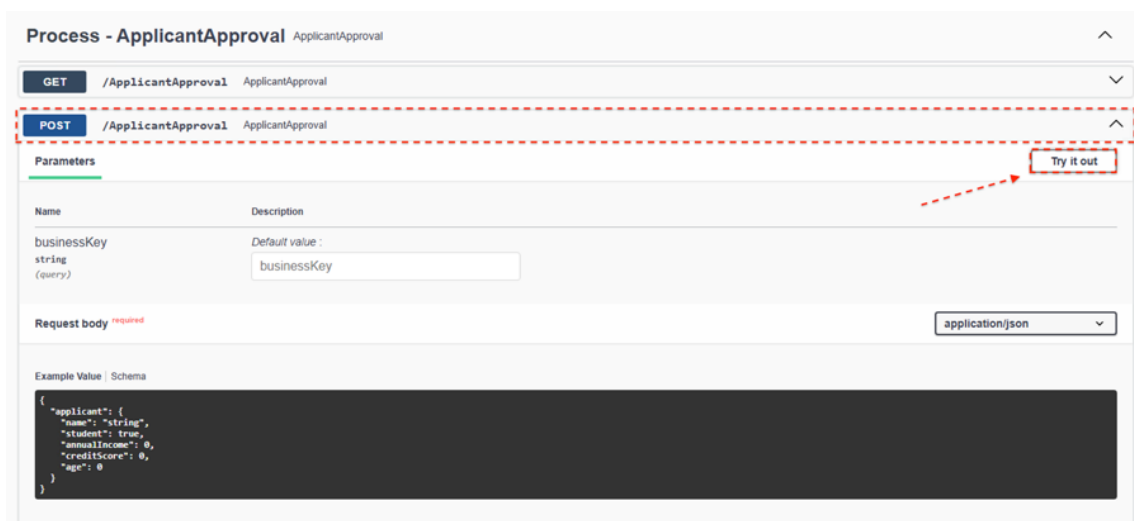
- Wait for the build to complete, open the Chrome browser, and access the Quarkus Dev UI: <http://localhost:8080/q/dev-ui>
- You can access the database tables by accessing the **"Database View"** in Agroal Extension.



- But before exploring the tables, let's use Swagger UI to create a few process instances. So, look for the **"SmallRye OpenAPI"** extension. Click on **"Swagger UI"**.



- Search for **"Process - Applicant Approval"**, we will use the **"POST"** method. Next, click **"Try it out"**.

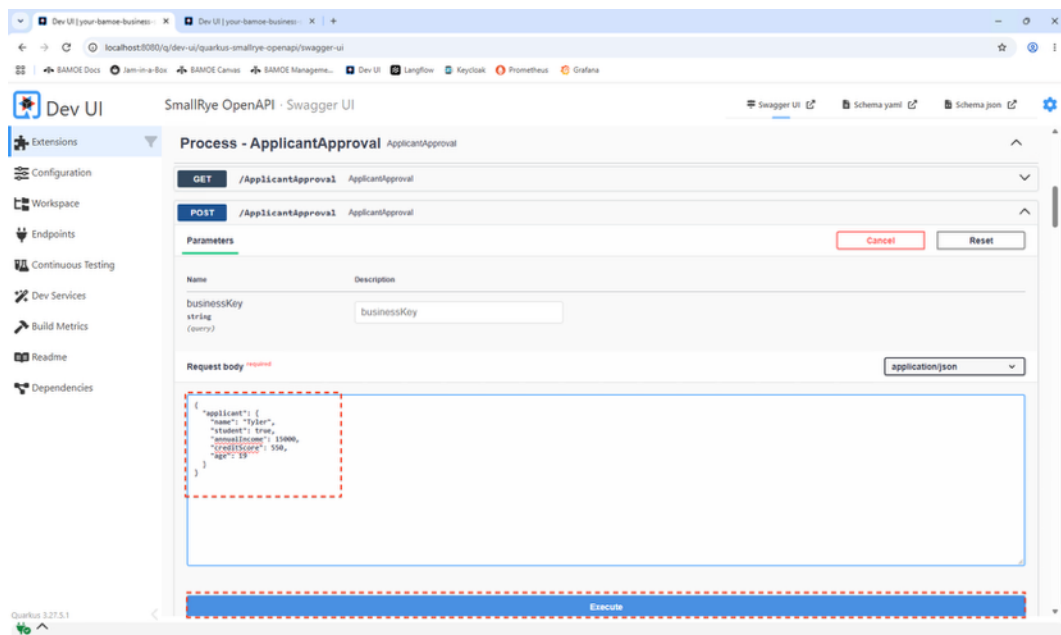


- Run these examples (one at a time) to populate the database.

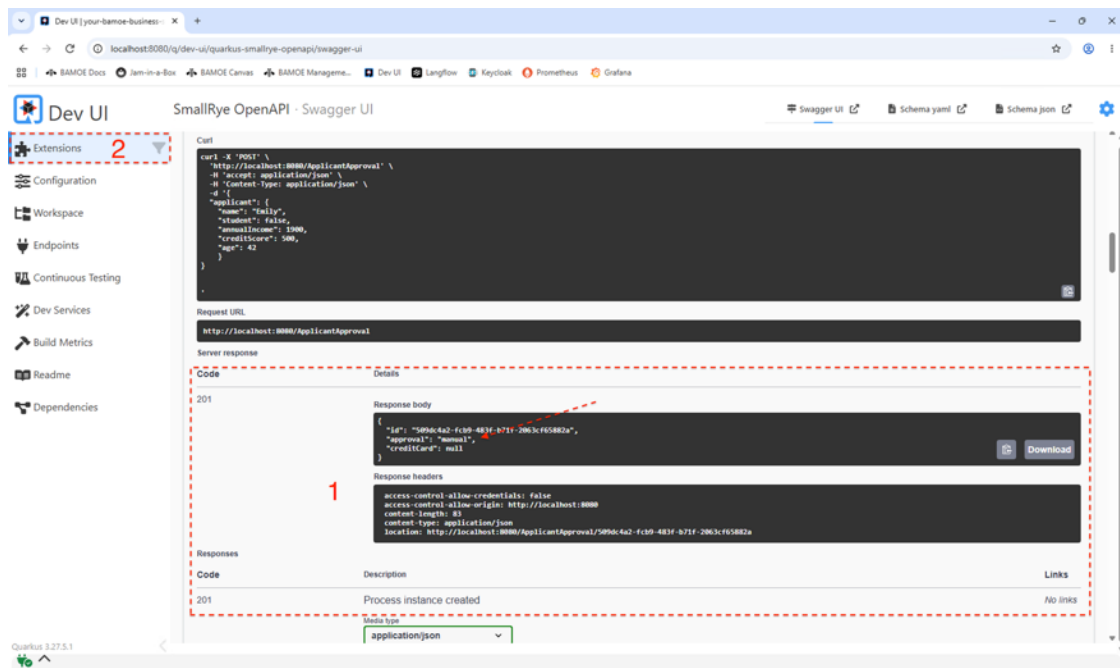
```
{
  "applicant": {
    "name": "Tyler",
    "student": true,
    "annualIncome": 15000,
    "creditScore": 550,
    "age": 19
  }
}
```

```
{
  "applicant": {
    "name": "David",
    "student": false,
    "annualIncome": 12000,
    "creditScore": 820,
    "age": 25
  }
}
```

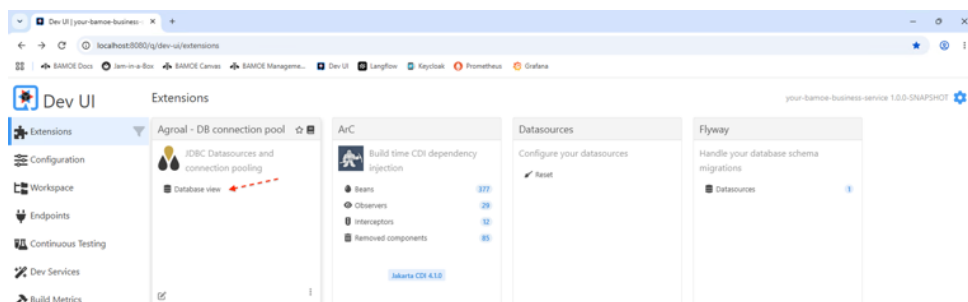
```
{
  "applicant": {
    "name": "Emily",
    "student": false,
    "annualIncome": 19000,
    "creditScore": 500,
    "age": 42
  }
}
```



- After running your examples, observe the returned Workflow result; in this use case, when "approval" results in "manual," a user task is created. You will then see this reflected in the database tables.



- Now, access the **"Agroal > Database View"** extension and explore some of the database tables.



- Explore the tables related to **Processes** and **Tasks**, and view the tests you executed.

The screenshot shows the Dev UI interface for a database connection pool. On the left, a sidebar lists various database tables. The 'processes' table is highlighted with a red dashed box. On the right, a detailed view of the 'processes' table is shown, displaying columns: id, business_key, end_time, and endpoint. The table contains four rows of data, with the first three rows highlighted by a red dashed box.

id	business_key	end_time	endpoint
a9874899-c541-4010-9700-71bdef7aee4b	null	2026-09-25 15:45:42.597	http://localhost:8080/
58627f36-215b-40cb-ba5a-c000830af4c	null	2026-09-25 15:49:29.529	http://localhost:8080/
06d236ba-3a4d-48a7-ae5e-b963882fc015	null	2026-09-25 15:50:32.915	http://localhost:8080/
509dc4a2-fcb9-483f-b71f-2063cf65882a	null	null	http://localhost:8080/

In the next exercise, we will improve this visualization using a Grafana dashboard. However, at this point, your Business Service Project is already storing and maintaining process data in your database. We are ready to move on to the next exercise.

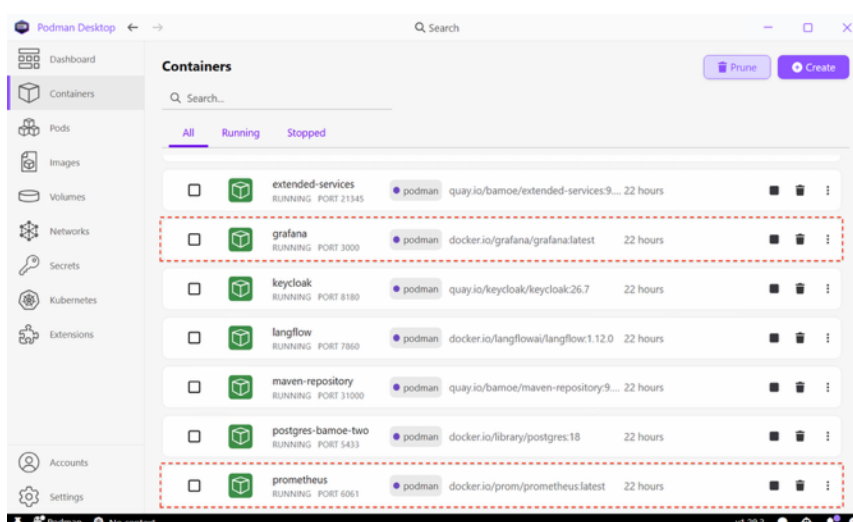
3.3 Exercise 2: Configure Grafana and Prometheus for monitoring and observability

If you have never worked with these tools before, a simple way to summarize and define them is to view Prometheus as a "data collector," while Grafana is what displays that data in a user-friendly way for daily use. They form a dynamic duo (known as the "Observability Stack") because they have complementary roles and do not try to do each other's job:

- Prometheus excels at fetching, storing, and performing complex calculations with numbers (metrics), but its visual interface is very rudimentary.
- Grafana is amazing for creating beautiful graphs and dashboards, but it lacks the ability to collect or store your application's data.

When you connect them, Grafana uses Prometheus as its data source.

Continuing with the setup, both containers are already deployed in your environment, Grafana on port 3000 and Prometheus on port 6061.

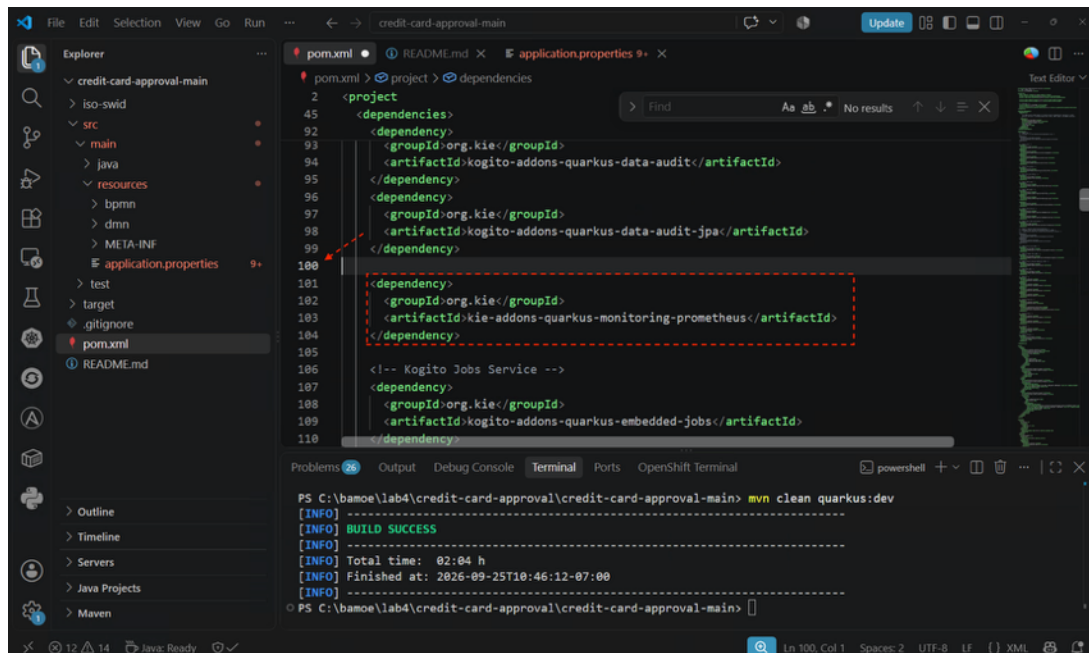


Note: You can find this and more information in [Monitoring Business Services](#).

- The first step for this configuration is to add the Prometheus dependency to the pom.xml file. I recommend stopping the project execution before you begin.
 - Simply press **H**, then **Q**, in the terminal.

- Open the **pom.xml** file and add the following dependency on line **100**:

```
<dependency>
  <groupId>org.kie</groupId>
  <artifactId>kie-addons-quarkus-monitoring-prometheus</artifactId>
</dependency>
```

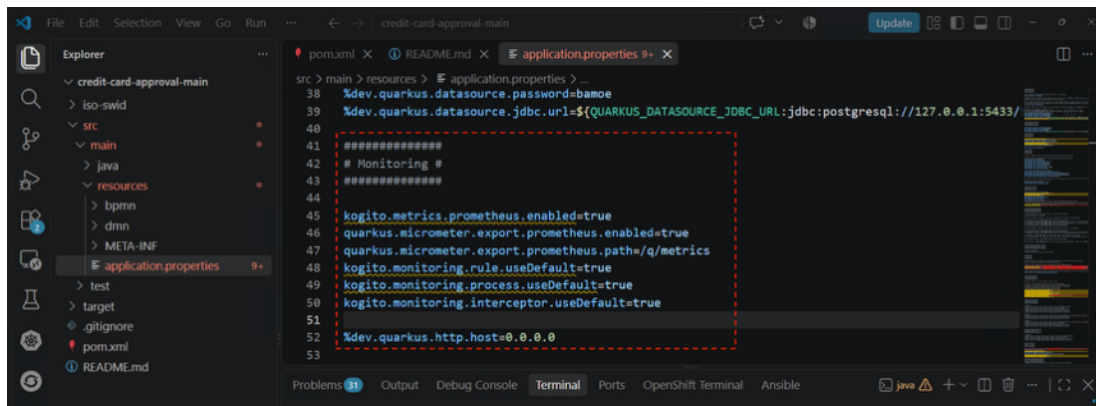


- Save the file.
- Now we need to enable the export of Business Services Project metrics to Prometheus.
- So open the **application.properties** file. Choose a line to include these settings (I placed them right after the Development database settings):

```
#####
# Monitoring #
#####

kogito.metrics.prometheus.enabled=true
quarkus.micrometer.export.prometheus.enabled=true
quarkus.micrometer.export.prometheus.path=/q/metrics
kogito.monitoring.rule.useDefault=true
kogito.monitoring.process.useDefault=true
kogito.monitoring.interceptor.useDefault=true

%dev.quarkus.http.host=0.0.0.0
```



- Save the file.
- Now that the BAMOE Business Service Project is configured to export metrics, I want you to understand how to configure Prometheus to retrieve them.
 - Open the Prometheus configuration file located at C:\bamoe\prometheus.yml
 - Note that this file contains the path to the Business Service metrics. The only change we need to make here is to replace "**targets**" with "**172.20.128.1:8080**".

```
C: > bamoe > ! prometheus.yml > {} scrape_configs > {} 0 > {} static_configs > {} 0 > {} targets > abc 0
prometheus.json - Prometheus configuration file (prometheus.json)
1  global:
2    scrape_interval: 5s
3  scrape_configs:
4    - job_name: 'bamoe-quarkus-app'
5      metrics_path: '/q/metrics'
6      static_configs:
7        - targets: ['172.20.128.1:8080']
```

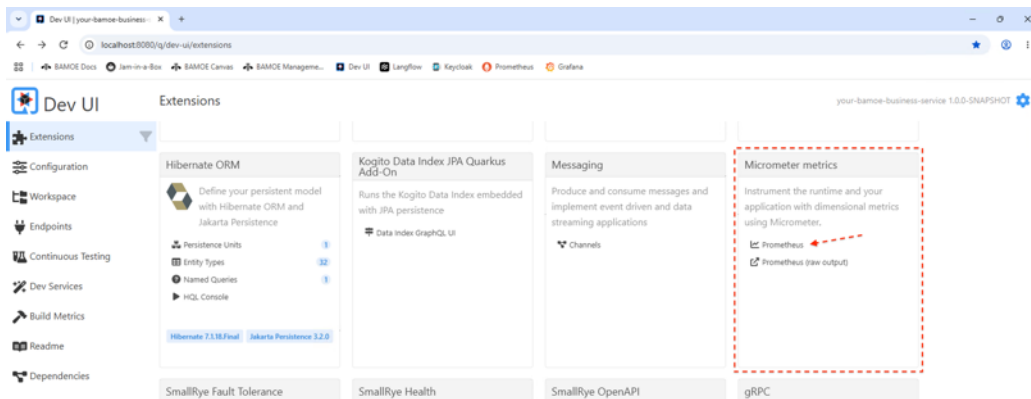
Why switch to using an IP address?

Since we are using Windows, Podman requires a Linux virtual machine, in this case, WSL2.

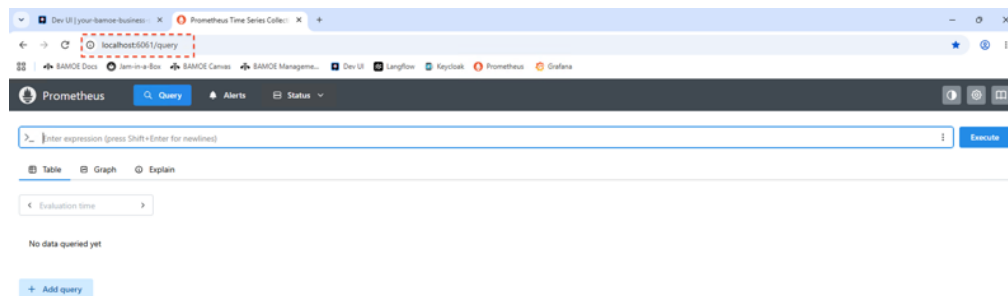
Therefore, to establish communication between the Windows host and the Podman container, we need to use the local machine's IP address.

Note – For techzone environments, you may use the same IP Address, however, if running on your own environment, retrieve your IP address and add the same here.

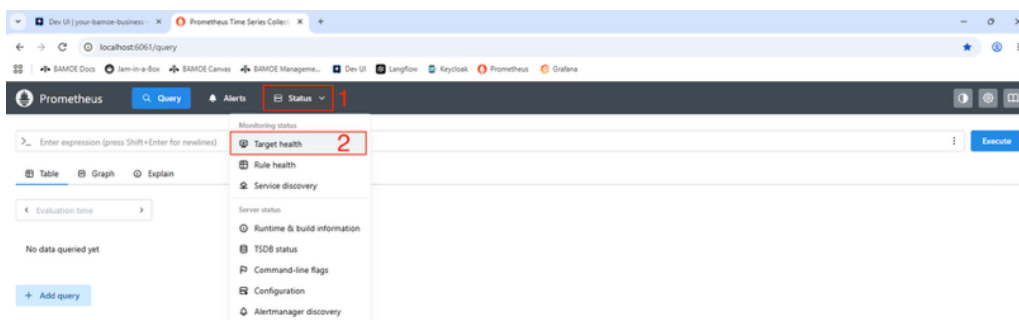
- Ensure that all files have been saved, and let's run the project. In the terminal, run:
`mvn quarkus:dev`
- Once the build completes, you will have access to the exposed metrics in the Quarkus Dev UI Extensions.
- Open the Chrome browser and go to: <http://localhost:8080/q/dev-ui/extensions>



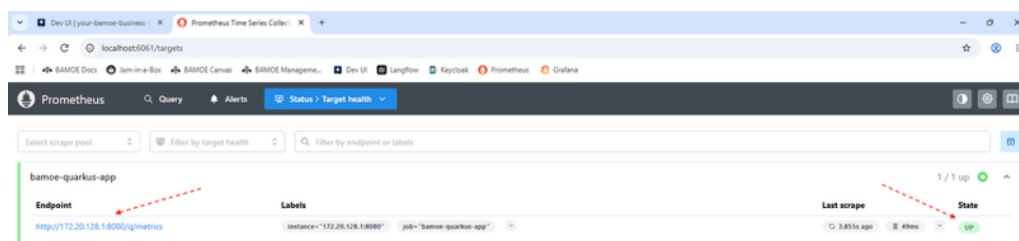
- In a new tab, open Prometheus: <http://localhost:6061/query>



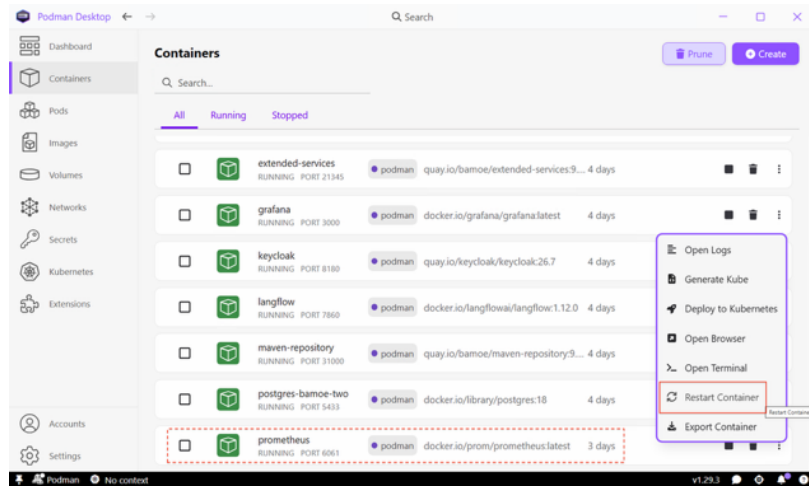
- Let's confirm that the connection was successful, then go to ***"Status > Target health"***



- You should get the same result: ***"State = Up"***



- If that doesn't happen, try restarting the Prometheus container:



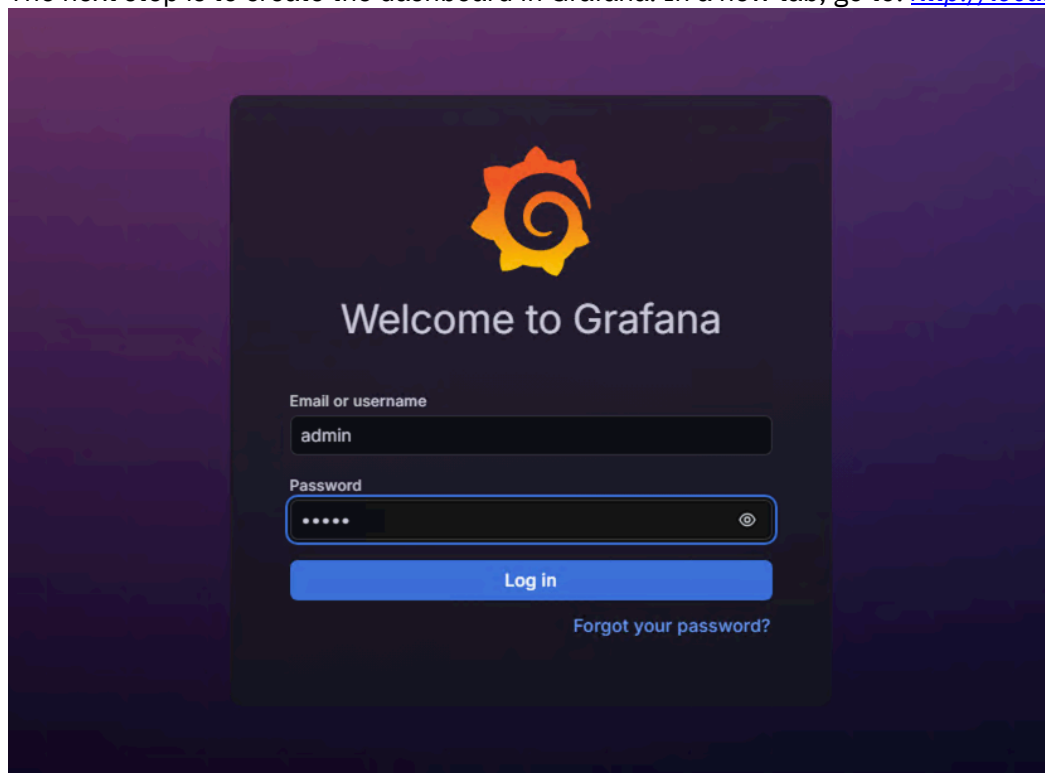
- Before we start building the dashboard, let's run the three tests again via Swagger UI to generate metrics for the Grafana dashboard.
- So go to <http://localhost:8080/q/dev-ui/quarkus-smallrye-openapi/swagger-ui>
- Run the tests:

```
{
  "applicant": {
    "name": "Tyler",
    "student": true,
    "annualIncome": 15000,
    "creditScore": 550,
    "age": 19
  }
}
```

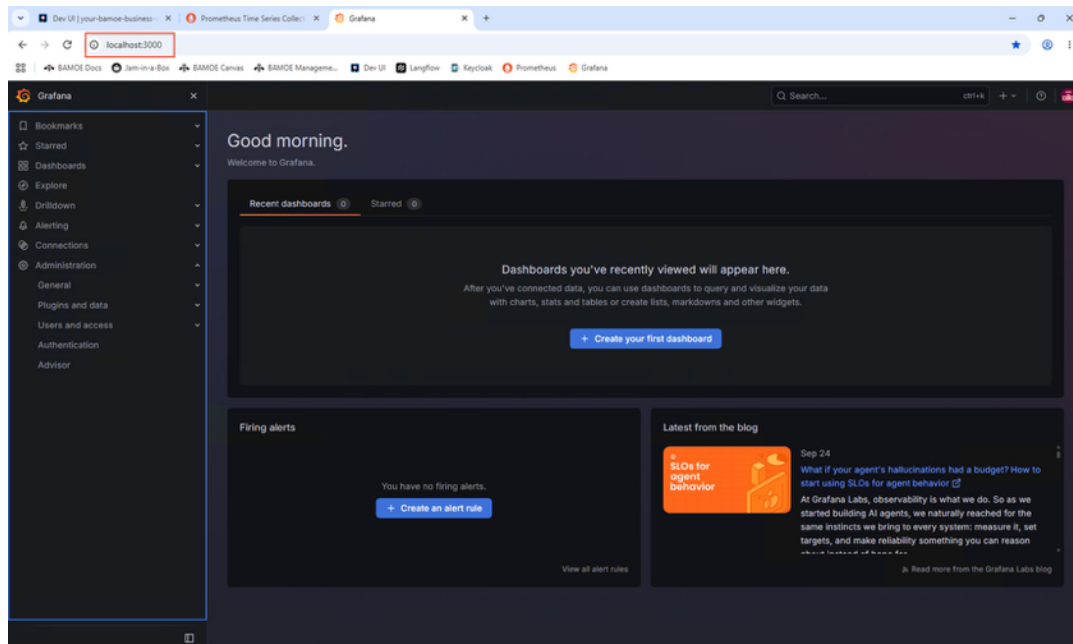
```
{
  "applicant": {
    "name": "David",
    "student": false,
    "annualIncome": 12000,
    "creditScore": 820,
    "age": 25
  }
}
```

```
{
  "applicant": {
    "name": "Emily",
    "student": false,
    "annualIncome": 19000,
    "creditScore": 500,
    "age": 42
  }
}
```

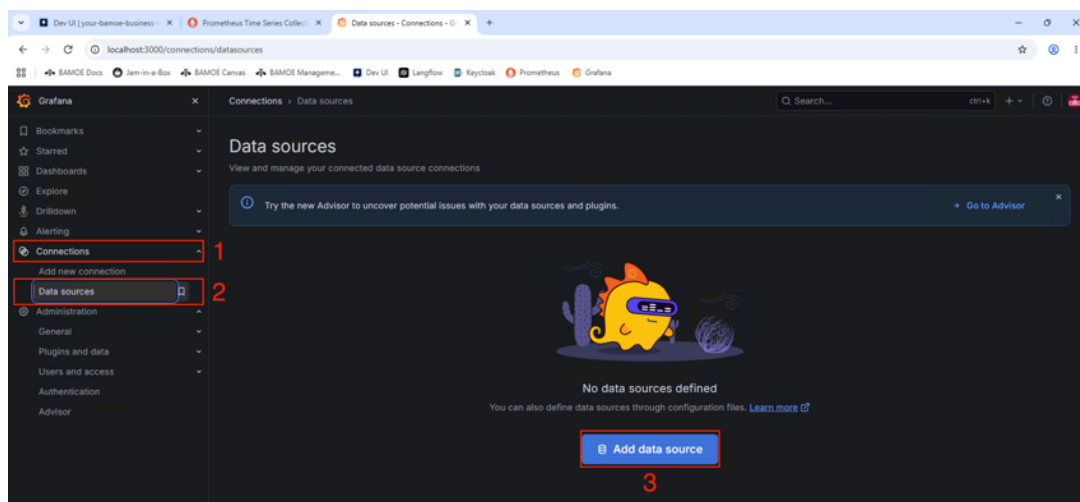
- The next step is to create the dashboard in Grafana. In a new tab, go to: <http://localhost:3000/>.



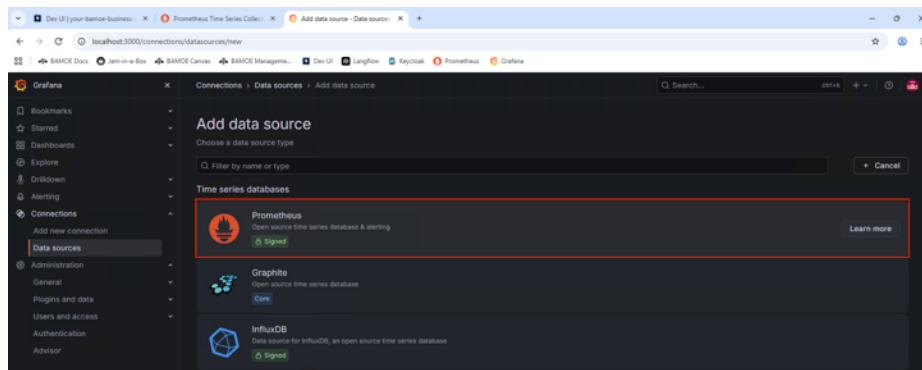
- The login screen appears, enter **username** – admin and **password** – admin and click on **Log in**.



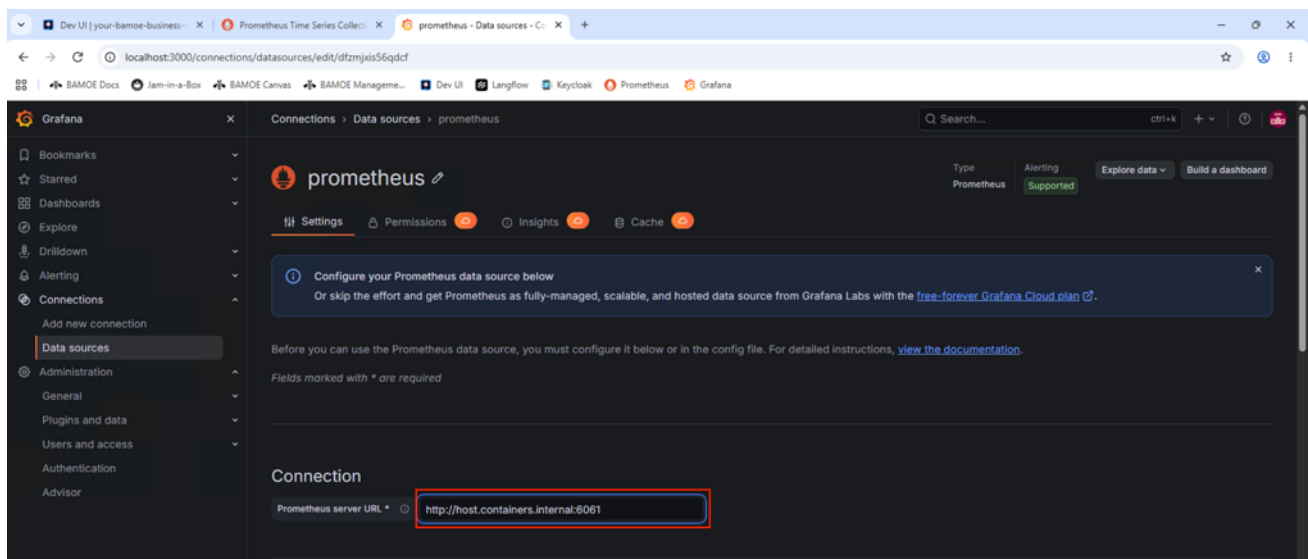
- Before creating the dashboard, let's add Prometheus as a data source. So, in the side menu, go to **“Connections > Data Sources”**. Click the **“Add data source”** button.

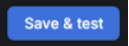


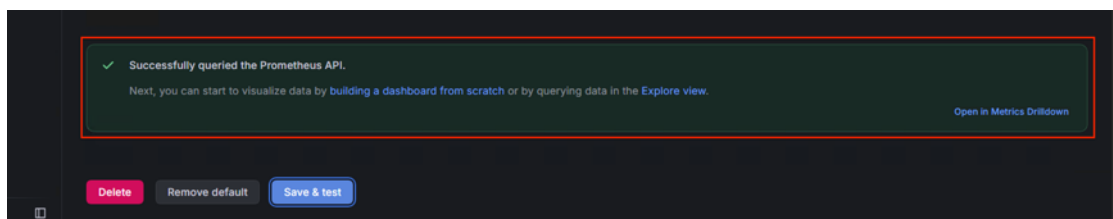
- Select “Prometheus



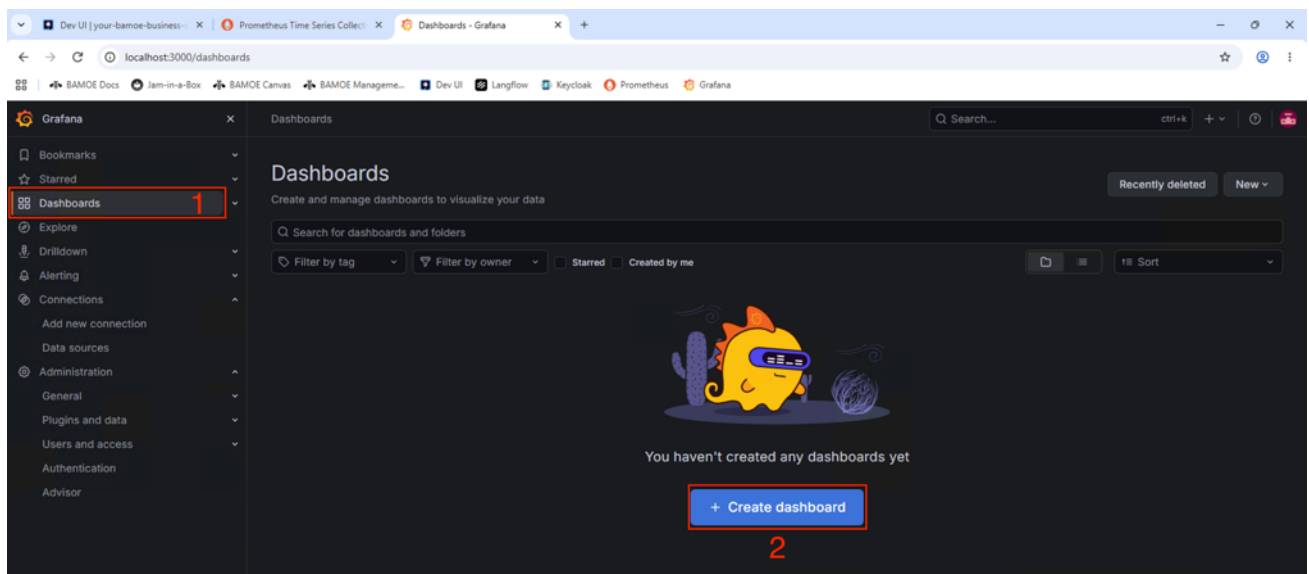
- Under "Connection," enter the Prometheus URL: <http://host.containers.internal:6061>



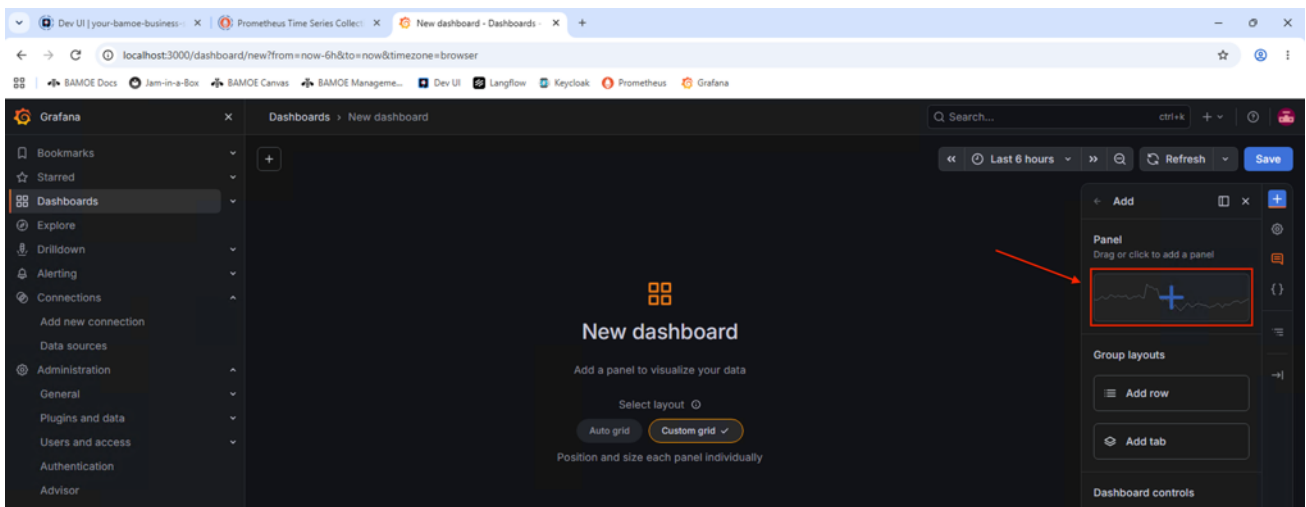
- Scroll down and click the **"Save & Test"** button .
- You should receive the success message.



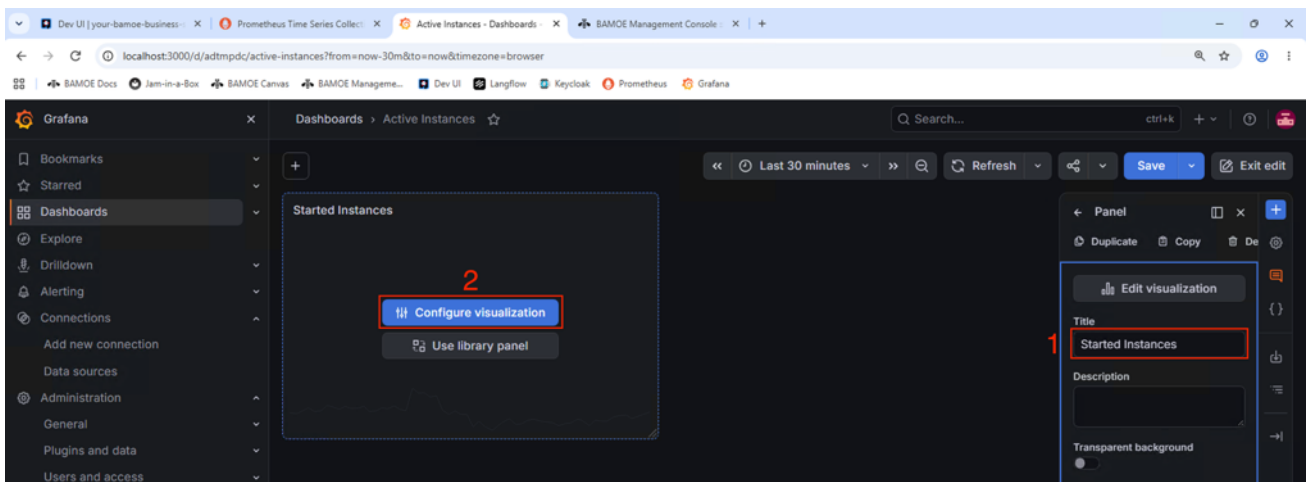
- Now, let's create a simple dashboard. In the side menu, go to Dashboard and click the **"Create your first dashboard"** button.



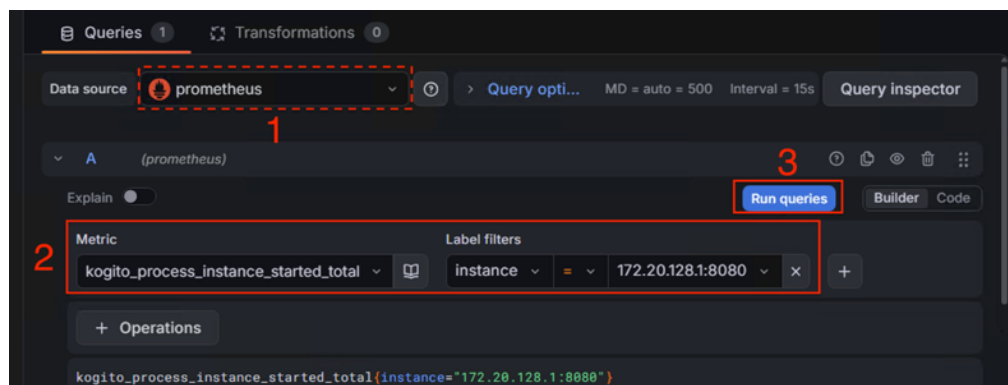
- In the right-hand panel, add a new **"Panel"**.



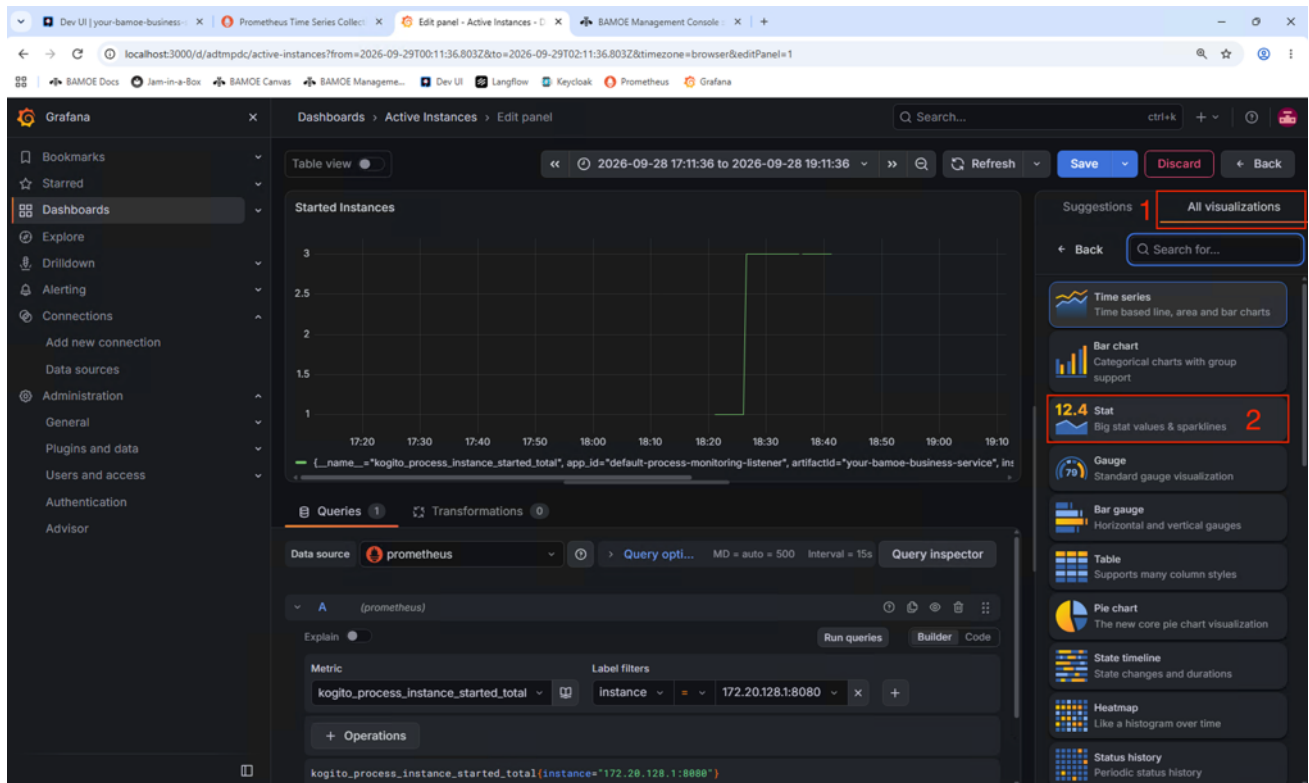
- Change the **"Title"** field to **"Started Instances"**, then, click the **"Configure visualization"** button.



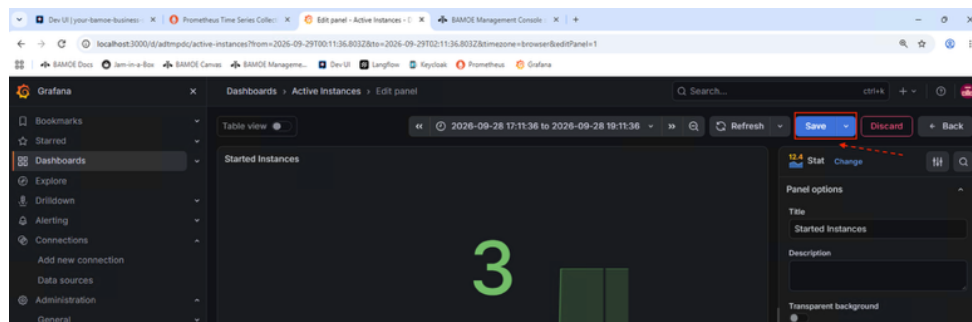
- Now, let's point to the metric that shows the started instances. So, ensure that **"prometheus"** is selected as the **data source**.
- Under **"Metric"** select **"kogito_process_instance_started_total"**, under **"Label filters"** select **"instance"**, and finally select the Business Service address.
- Click the **"Run queries"** button.



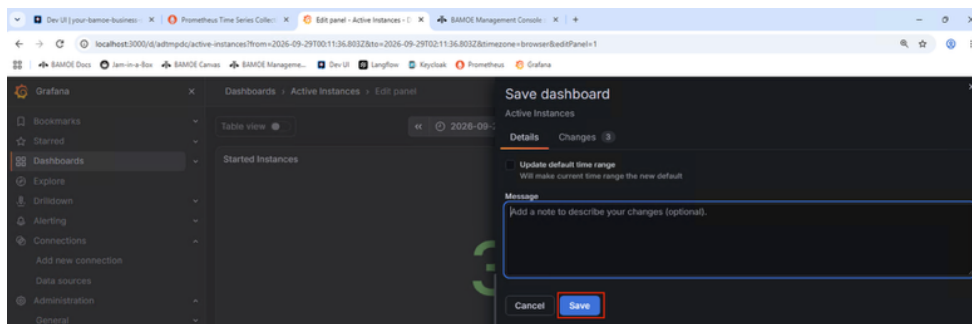
- I recommend using the **"Stat"** view to make it easier to check the tests, but feel free to try out other views later.



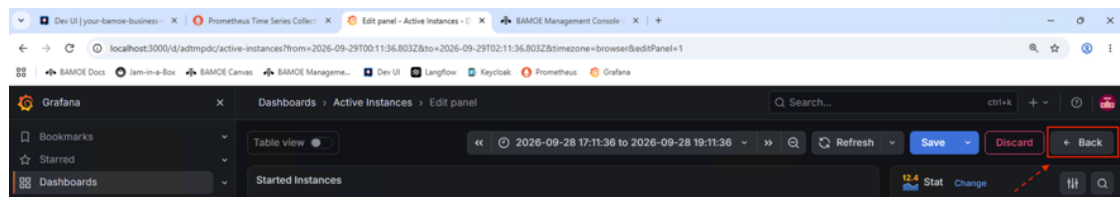
- Click **"Save"**



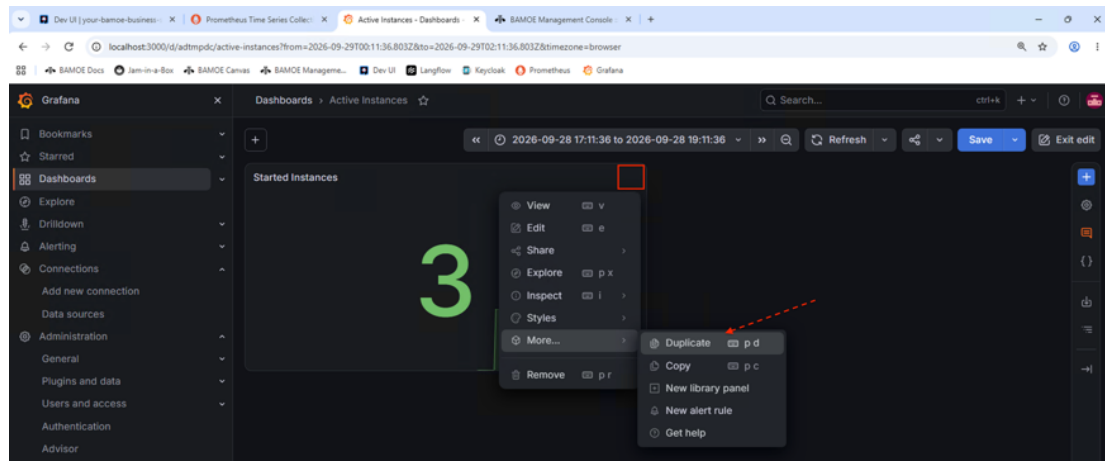
- Click **"Save"** again



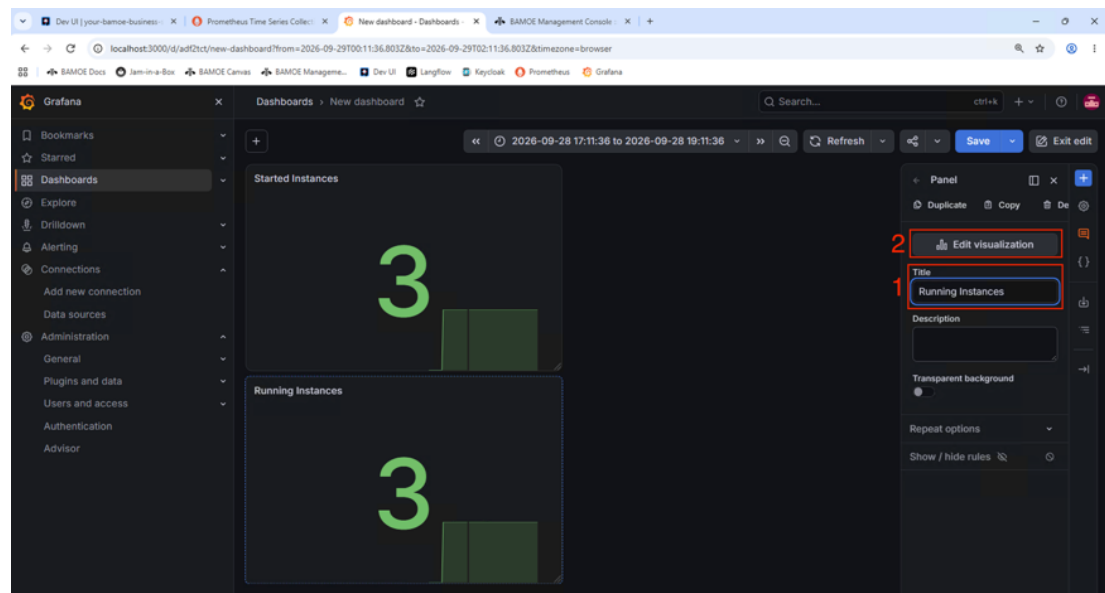
- Click the **"Back"** button to return to the Dashboard.



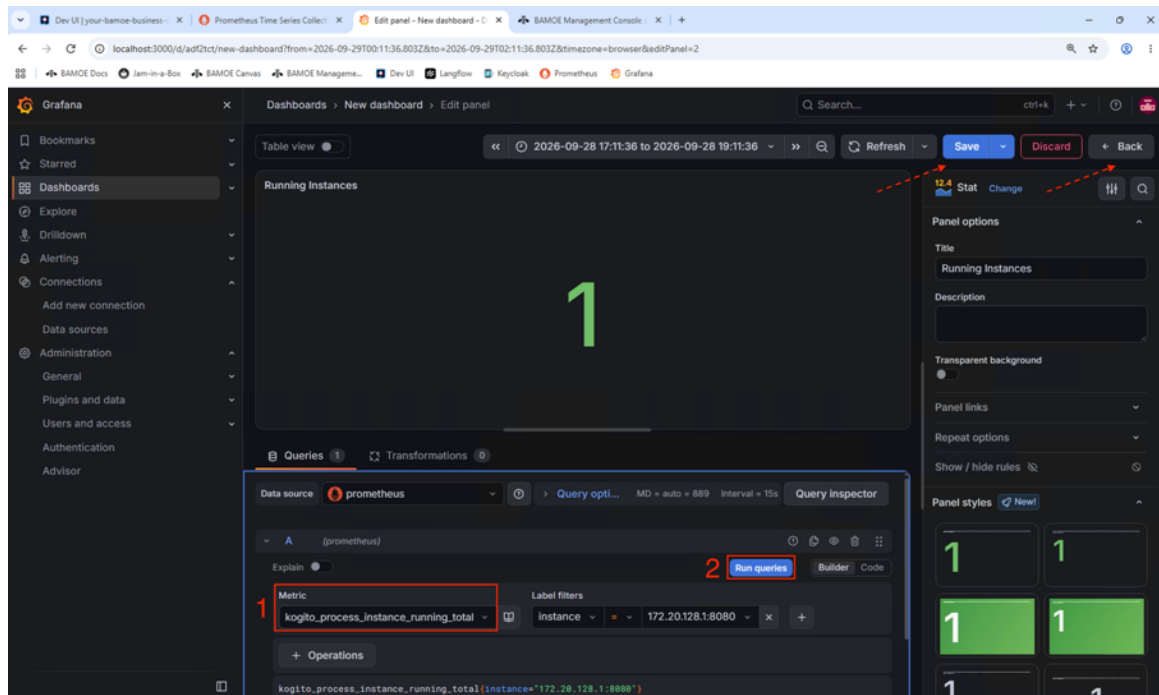
- To add the other panels, I think it's easiest to duplicate the one we just created. So, click the button in the top right of the panel > **"More..."** > **"Duplicate"**.



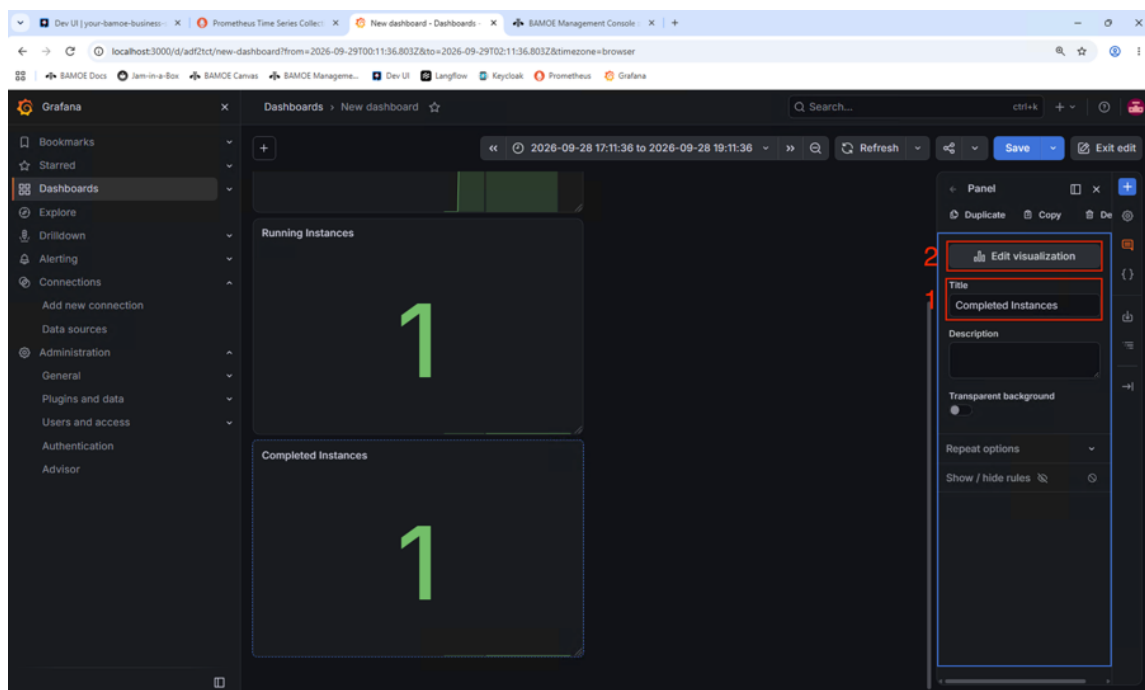
- Let's call this panel **"Running Instances"**. Click **"Edit visualization"** to change the metric.



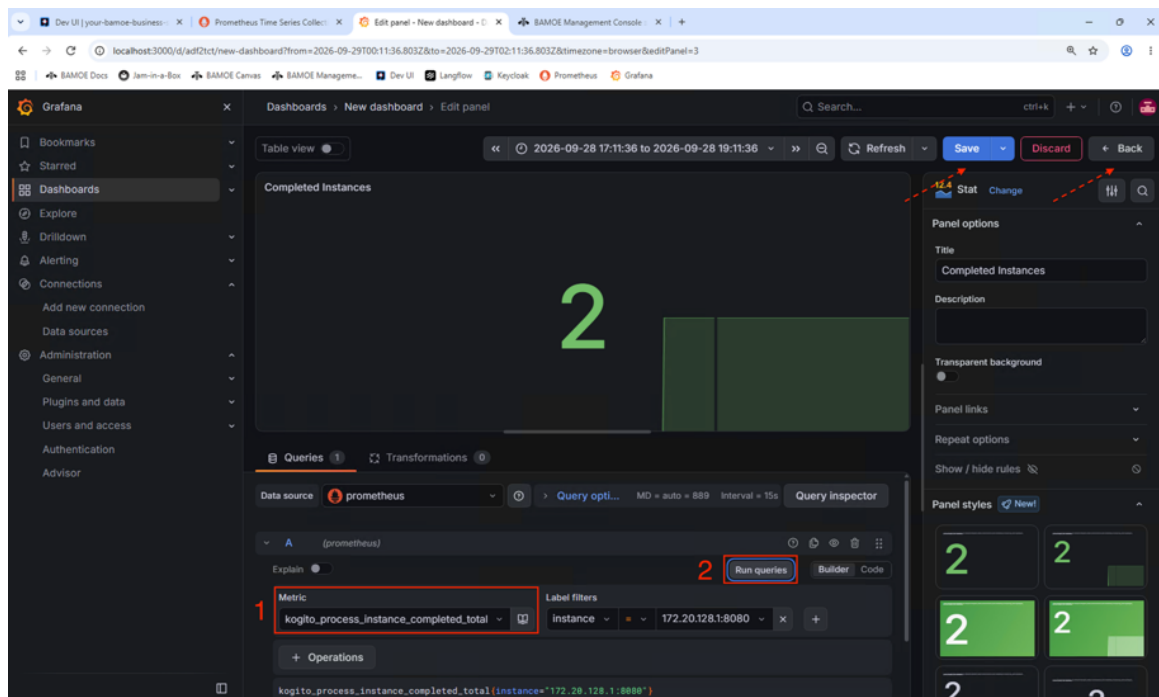
- Change the metric, select **"kogito_process_instance_running_total"**, click the **"Run queries"** button, then save and return to the dashboard.



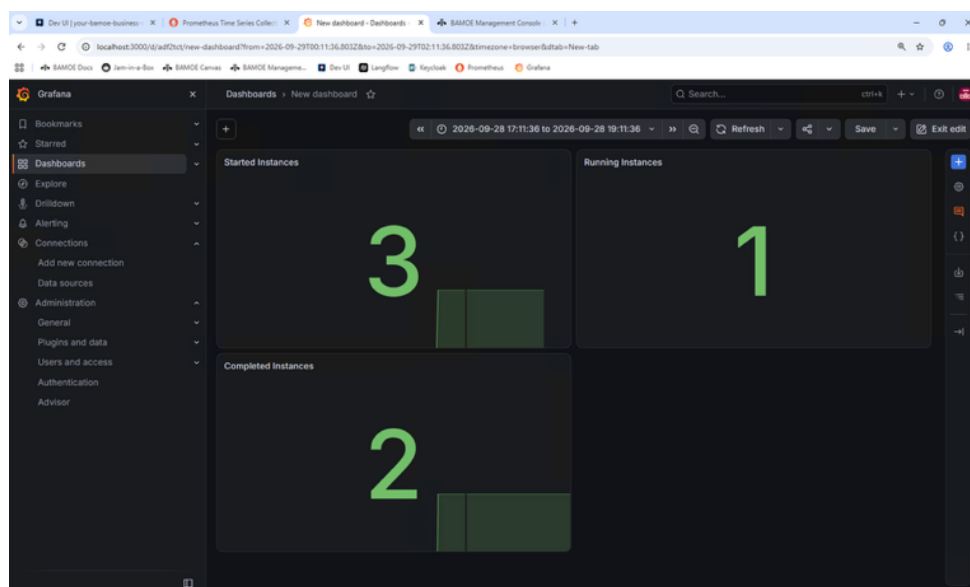
- Duplicate another one and name it **"Completed Instances"**.



- Change the metric, select **"kogito_process_instance_completed_total"**, click the **"Run queries"** button, then save and return to the dashboard.



- I recommend exploring Grafana further to create new visualizations. Test other metrics, and even create new instances to analyze the dashboard.
- Consider the information we have:
 - Started Instances:** The total number of instances started, basically the three tests we ran at the beginning.
 - Running Instances:** Here we have the number of instances that are still running. Returning to our tests, only one case requires manual approval, and since we haven't completed that user task yet, it is still in progress.
 - Completed Instances:** Here are all the ones that already have a status of "completed."



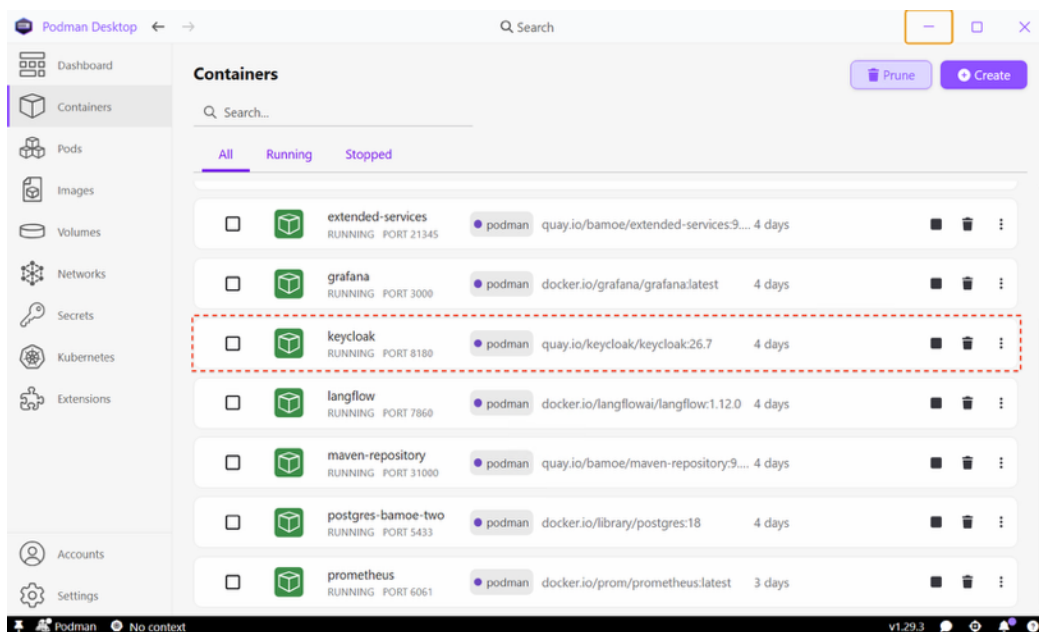
We'll wrap up this topic here.

3.4 Exercise 3: Configure Keycloak for security and user management

In this exercise, we will demonstrate how to configure an external Identity Provider (IdP) to integrate a security layer into your project's Identity and Access Management solution.

The BAMOE Management Console can connect to secured Business Services, provided they expose sufficient information about the configured Identity Provider. In this case, we will be using Keycloak, an open-source tool widely used in large-scale projects.

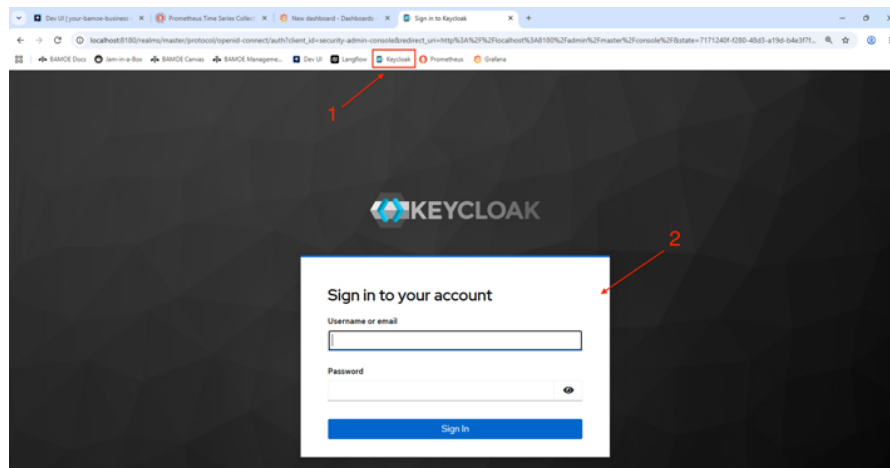
Following the same structure, your environment now has the Keycloak Podman container running; you can access it at <http://localhost:8180>.



Note: You can find this and more information in [Securing Business Service API endpoints](#).

3.4.1 Step 01: Configuring the Keycloak application

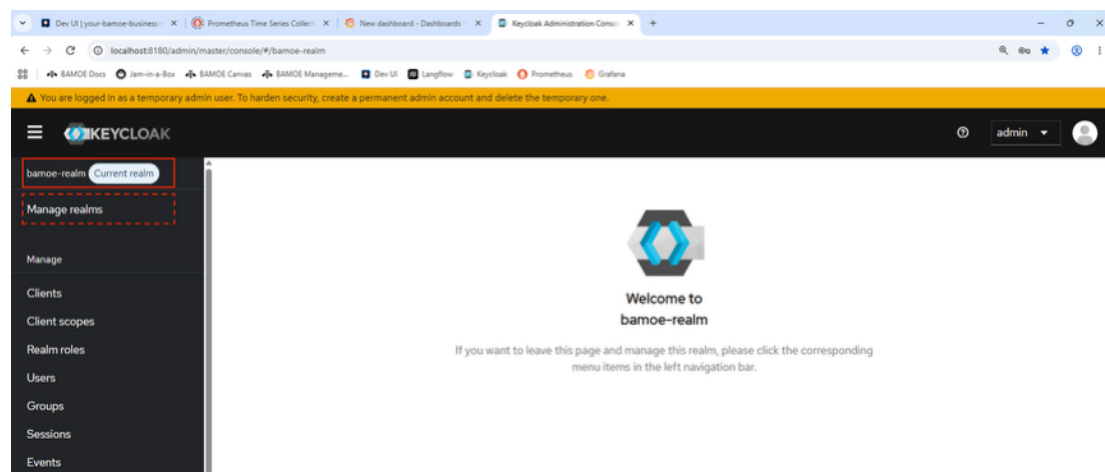
- Let's explore Keycloak and begin its configuration.
- Access <http://localhost:8180/admin/master/console/#/bamoe-realm> in your browser and log in.
 - Username: `admin`
 - Password: `admin`



To begin with, we need to perform 4 configurations:

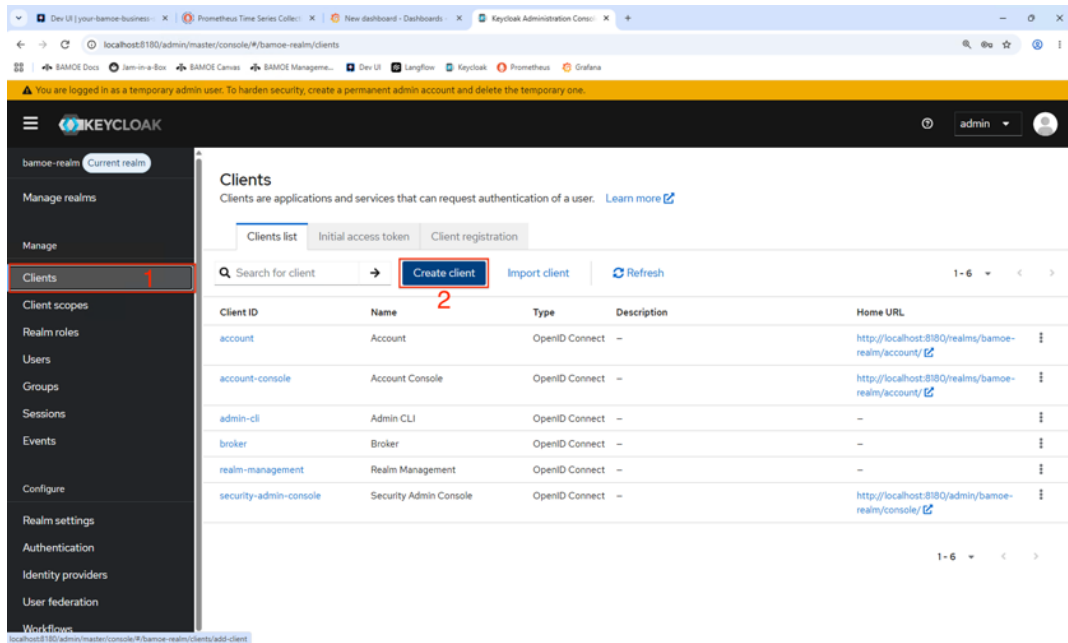
1. Manage realm:

- Create a realm for our BAMOE project. Registration is simple, and we have already set it up in this environment. So, just ensure that **"bamoe-realm"** is selected.
- But if it isn't, click **"Manage realms"** and select it.



2. Client Interface:

- We need to create one client for the user interface and another for the service.
- In the side menu, click on **"Clients" > "Create client"**



- Keep **"Client type"** set to **"OpenID Connect"** and enter **"bamoe-management-console"** in the **"Client ID"** field.
- Click the **Next** button.

The screenshot shows the 'Create client' form in the Keycloak Administration Console. The form is titled 'Create client' and has a subtitle 'Clients are applications and services that can request authentication of a user.' Below the title, there is a 'General settings' section. In this section, the 'Client type' dropdown is set to 'OpenID Connect' and the 'Client ID' field contains 'bamoe-management-console'. The 'Name' and 'Description' fields are empty. The 'Always display in UI' toggle is turned off. At the bottom of the form, there are three buttons: 'Back', 'Next', and 'Cancel'. The 'Next' button is highlighted with a red box.

- Keep the default settings and click **Next**.

1 General settings
2 **Capability config**

Client authentication ☐ Off

Authorization ☐ Off

Authentication flow ☒ Standard flow ☐ Direct access grants ☐ Service account roles ☐

☐ Implicit flow ☐ Standard Token Exchange ☐ JWT Authorization Grant ☐ OAuth 2.0 Device Authorization Grant ☐ OIDC CIBA Grant

Require PKCE ☐ Off

Require DPoP bound tokens ☐ Off

Back **Next** Cancel

- Add the BAMOE Management Console URL <http://localhost:8280/> to the **"Valid redirect URIs"** and **"Web origins"** fields.

Create client

Clients are applications and services that can request authentication of a user.

1 General settings
2 Capability config
3 **Login settings**

Root URL

Home URL

Valid redirect URIs [Add valid redirect URIs](#)

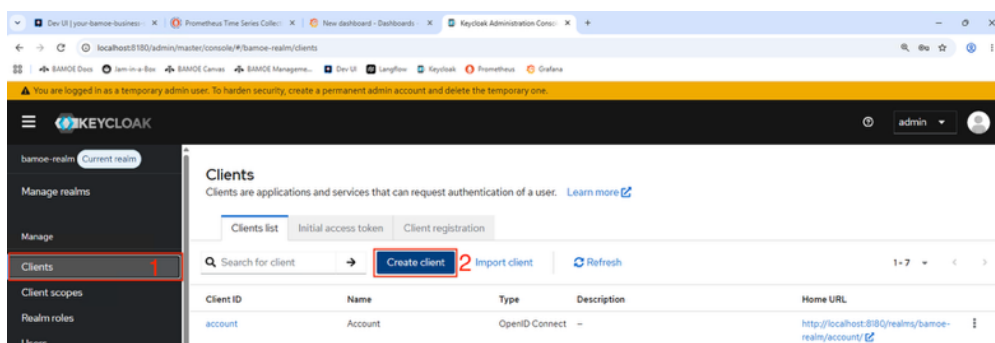
Valid post logout redirect URIs [Add valid post logout redirect URIs](#)

Web origins [Add web origins](#)

Back **Save** Cancel

3. Client Service:

- Let's create a client for the service.
- Return to the **"Client"** menu, then select **"Create client"**.



- Keep **"Client type"** set to **"OpenID Connect"** and enter **"bamoe-business-service"** in the **"Client ID"** field.
- Click the **Next** button.

Clients > Create client

Create client
Clients are applications and services that can request authentication of a user.

1 General settings

Client type ⓘ OpenID Connect

Client ID ⓘ bamoe-business-service

Name ⓘ

Description ⓘ

Always display in UI ⓘ Off

Back Next Cancel

- Enable **"Client authentication"** and also select the **"Service account roles"** option. Click **Next**.

Clients > Create client

Create client
Clients are applications and services that can request authentication of a user.

1 General settings

2 Capability config

1 Client authentication ⓘ On

Authorization ⓘ Off

Authentication flow

☒ Standard flow ⓘ

☐ Implicit flow ⓘ

☐ Standard Token Exchange ⓘ

☐ JWT Authorization Grant ⓘ

☐ OAuth 2.0 Device Authorization Grant ⓘ

☐ OIDC CIBA Grant ⓘ

☐ Direct access grants ⓘ

2 ☒ Service account roles ⓘ

Require PKCE ⓘ Off

Require DPoP bound tokens ⓘ Off

- Now you can save.

Create client

Clients are applications and services that can request authentication of a user.

- 1 General settings
- 2 Capability config
- 3 Login settings

Root URL ⓘ

Home URL ⓘ

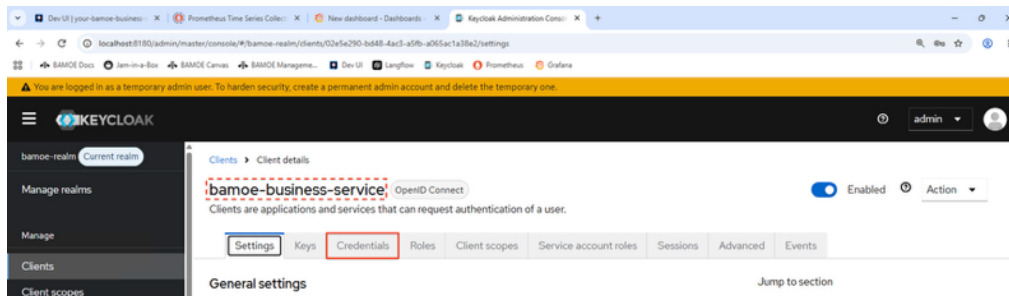
Valid redirect URIs ⓘ
Add valid redirect URIs

Valid post logout redirect URIs ⓘ
Add valid post logout redirect URIs

Web origins ⓘ
Add web origins

Back Save Cancel

- After saving, we need to obtain this client's secret, in the next steps, we need to enter it into the BAMOE Business Service Project.
- So, within the **"bamoe-business-service"** client, go to the **"Credentials"** tab.



- Copy the "Client Secret" and save it somewhere of your choice. Alternatively, you can come back here to copy it again whenever you need it.

Clients > Client details

bamoe-business-service OpenID Connect Enabled ⓘ Action

Clients are applications and services that can request authentication of a user.

Settings Keys Credentials Roles Client scopes Service account roles Sessions Advanced Events

Jump to section

Client Authenticator Client Id and Secret

Allowed authentication method Any

Client Secret ⓘ

Regenerate

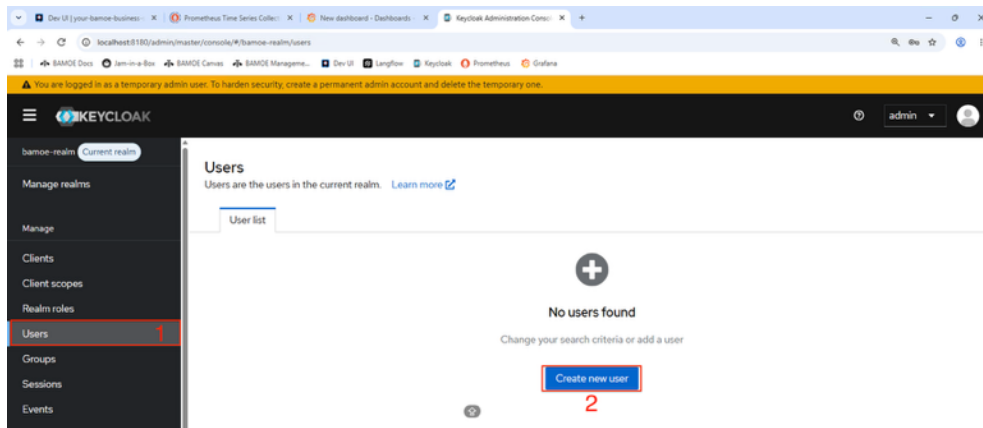
Save Revert

Registration access token ⓘ

Regenerate

4. User:

- Finally, let's create the username and password for the project's standard user, "Jdoe".
- Navigate to **"Users"** and click **"Create new user"**.



- Enter **"jdoe"** in the **"Username"** field. Then click **"Create"**.

Users > Create user

Create user

Required user actions: Select action

Email verified: ☐ Off

General

Username:

Email:

First name:

Last name:

Groups:

- Go to the **"Credentials"** tab and click **"Set password"**

Users > User details

jdoe Enabled Action

Details **Credentials** Role mapping Groups Consents Identity provider links Sessions Events Workflows

No credentials

This user does not have any credentials. You can set password for this user.

- Create a password and turn off the **"temporary"** option. I used **"jdoe"** for the password.

- Click **"Save password"**

We have finished the basic configurations in Keycloak. In the next step, we will configure the Business Service Project side.

3.4.2 Step 02: Configuring the BAMOE Business Service Project

Back to the open project in VS Code, I recommend stopping the project execution before starting the configuration of the “*pom.xml*” and “*application.properties*” files.

- Simply press **H**, then **Q**, in the terminal.

1. Dependencies in the *pom.xml* file

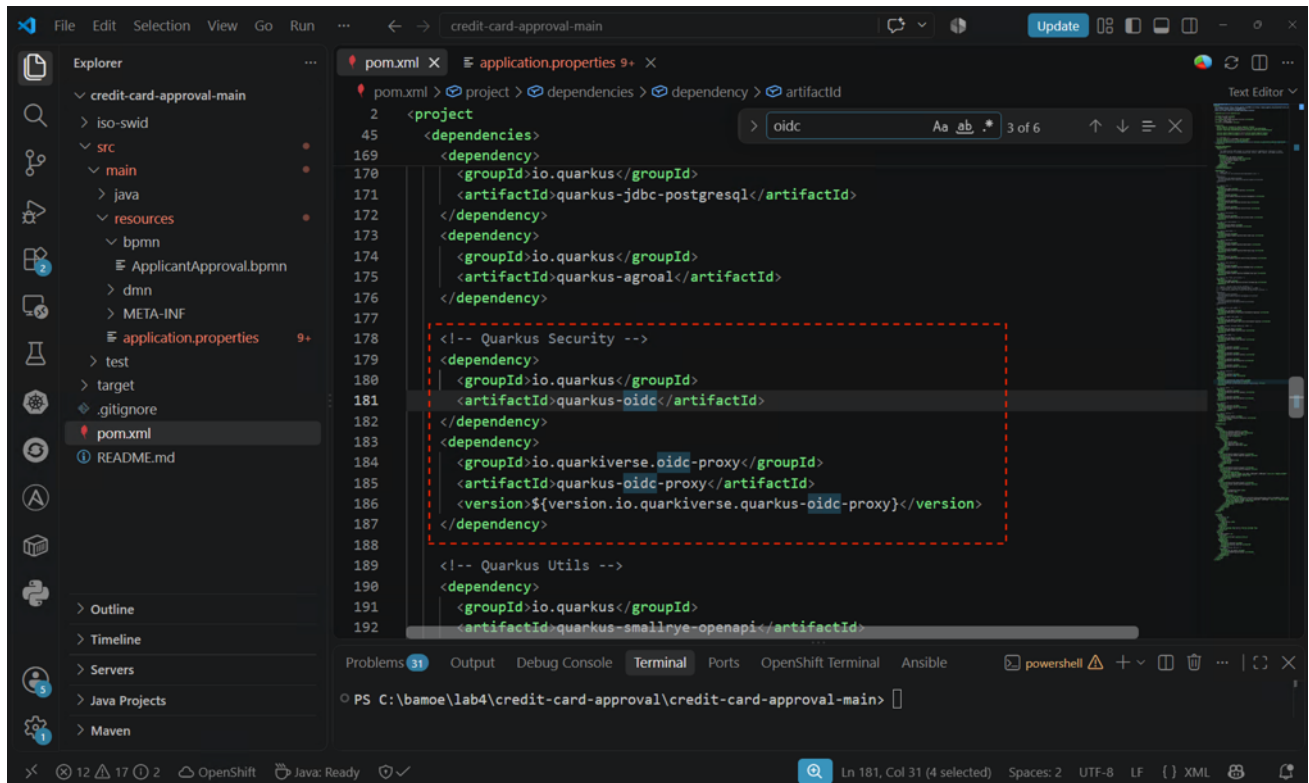
As previously mentioned, this project was created using a BAMOE accelerator for Workflows, so many dependencies and even configurations are already set up. But just so you know, to integrate your project with Keycloak, you need to have these three dependencies:

```
<!-- Quarkus Security -->
<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-oidc</artifactId>
</dependency>

<dependency>
  <groupId>io.quarkiverse.oidc-proxy</groupId>
  <artifactId>quarkus-oidc-proxy</artifactId>
  <version>${version.io.quarkiverse.quarkus-oidc-proxy}</version>
</dependency>

<!-- ADD DATA INDEX -->
<dependency>
  <groupId>org.kie</groupId>
  <artifactId>kogito-addons-quarkus-data-index-jpa</artifactId>
</dependency>
```

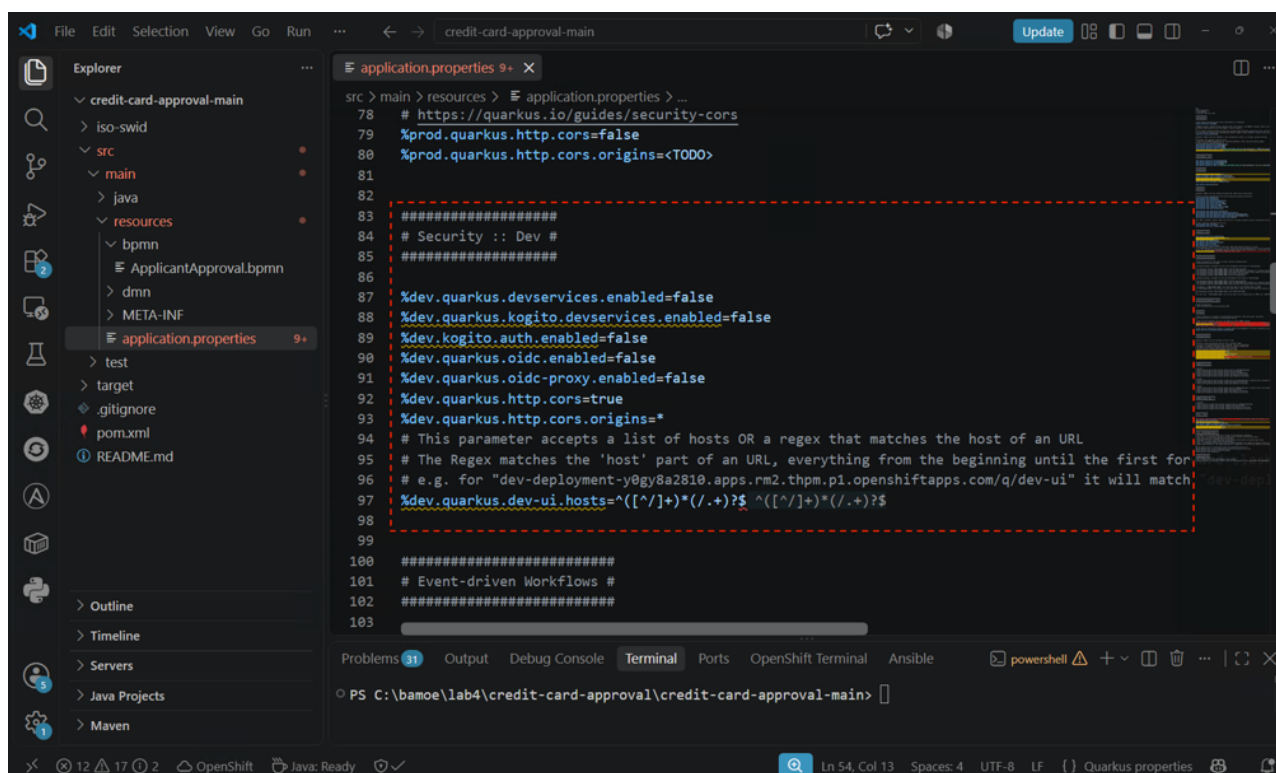
You can open your pom.xml file and search for each of them, and if they don't exist, add them.



2. Configure *application.properties*

Now, there are several configurations to set up in “**application.properties**”. Many of these are already pre-configured in the file, if you search for “**Security**” you will see various configuration blocks, some for development execution (%*dev*) and others for production (%*prod*).

- Since our goal is to test these features in development mode, go to the ***"Security :: Dev"*** block.



- Below, I have organized the settings and added comments to help explain each one. You can copy them and replace the **"Security :: Dev"** block.

```
%dev.kogito.auth.enabled=true
%dev.quarkus.oidc.enabled=true
%dev.quarkus.oidc.discovery-enabled=true
%dev.quarkus.oidc.tenant-enabled=true

# URL of the Keycloak realm we created for the BAMOE project
%dev.quarkus.oidc.auth-server-url=http://[::1]:8180/realms/bamoe-realm

# Identification of the Business Service Client we created in Keycloak in the previous
step
%dev.quarkus.oidc.client-id=bamoe-business-service
%dev.quarkus.oidc.credentials.secret= << PASTE THE SECRET HERE >>
%dev.quarkus.oidc-proxy.enabled=true
%dev.quarkus.oidc.application-type=service

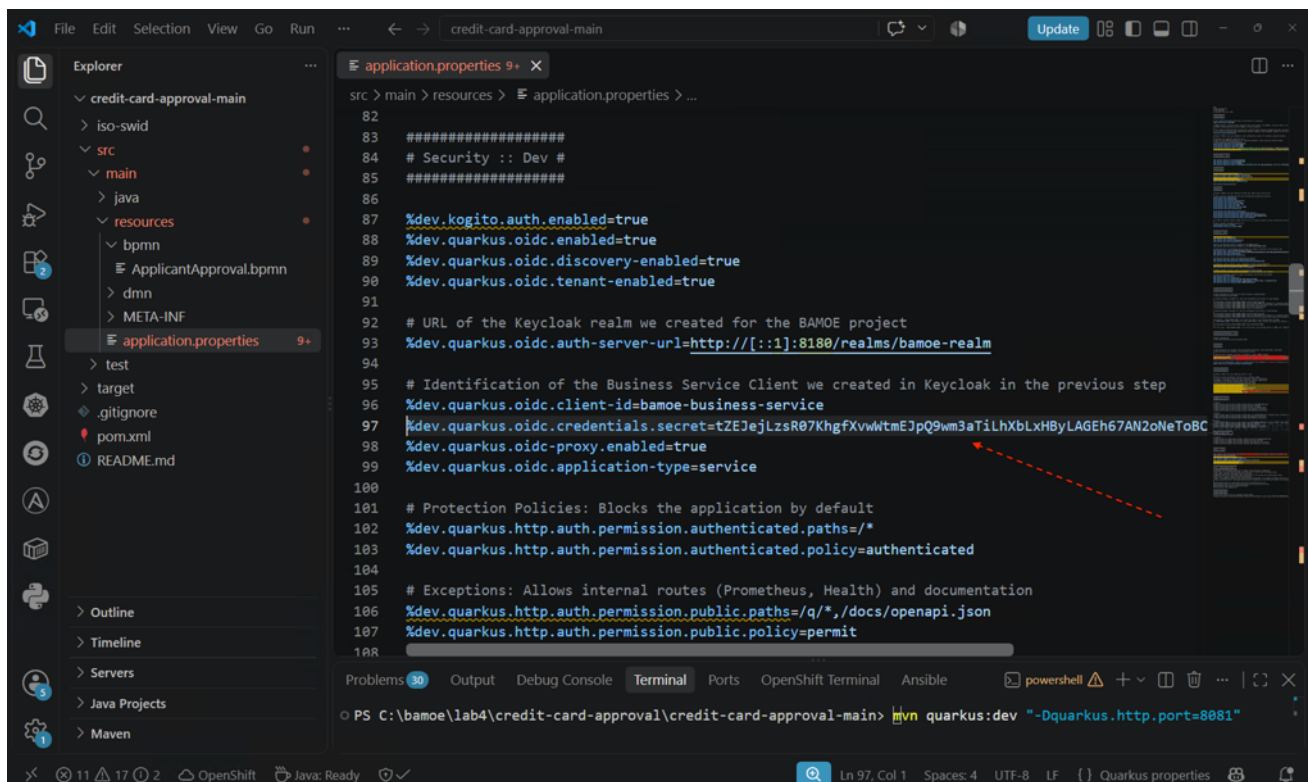
# Protection Policies: Blocks the application by default
%dev.quarkus.http.auth.permission.authenticated.paths=/*
%dev.quarkus.http.auth.permission.authenticated.policy=authenticated

# Exceptions: Allows internal routes (Prometheus, Health) and documentation
%dev.quarkus.http.auth.permission.public.paths=/q/*,/docs/openapi.json
%dev.quarkus.http.auth.permission.public.policy=permit
```

```
# Adding /graphql to public routes to facilitate initial testing
%dev.quarkus.http.auth.permission.public.paths=/q/*,/docs/openapi.json,/graphql

# Allows the Management Console to query the Business Service APIs and GraphQL
%dev.quarkus.http.cors=true
%dev.quarkus.http.cors.origins=.*/*
%dev.quarkus.http.cors.methods=GET,PUT,POST,DELETE,OPTIONS
%dev.quarkus.http.cors.headers=accept, authorization, content-type, x-requested-with
%dev.quarkus.http.cors.exposed-headers=WWW-Authenticate
```

- At this point, you need to enter the secret generated in Keycloak and include it in the **"%dev.quarkus.oidc.credentials.secret"** field.

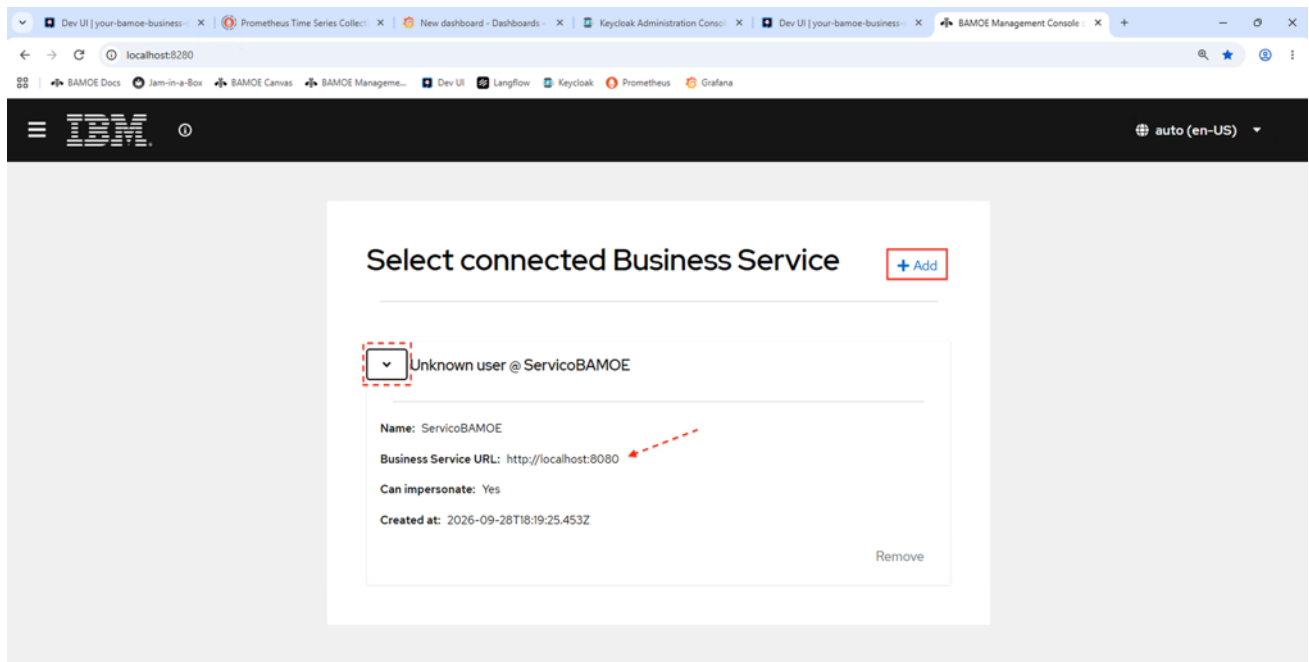


- Save the file, and let's run it to perform the final test.

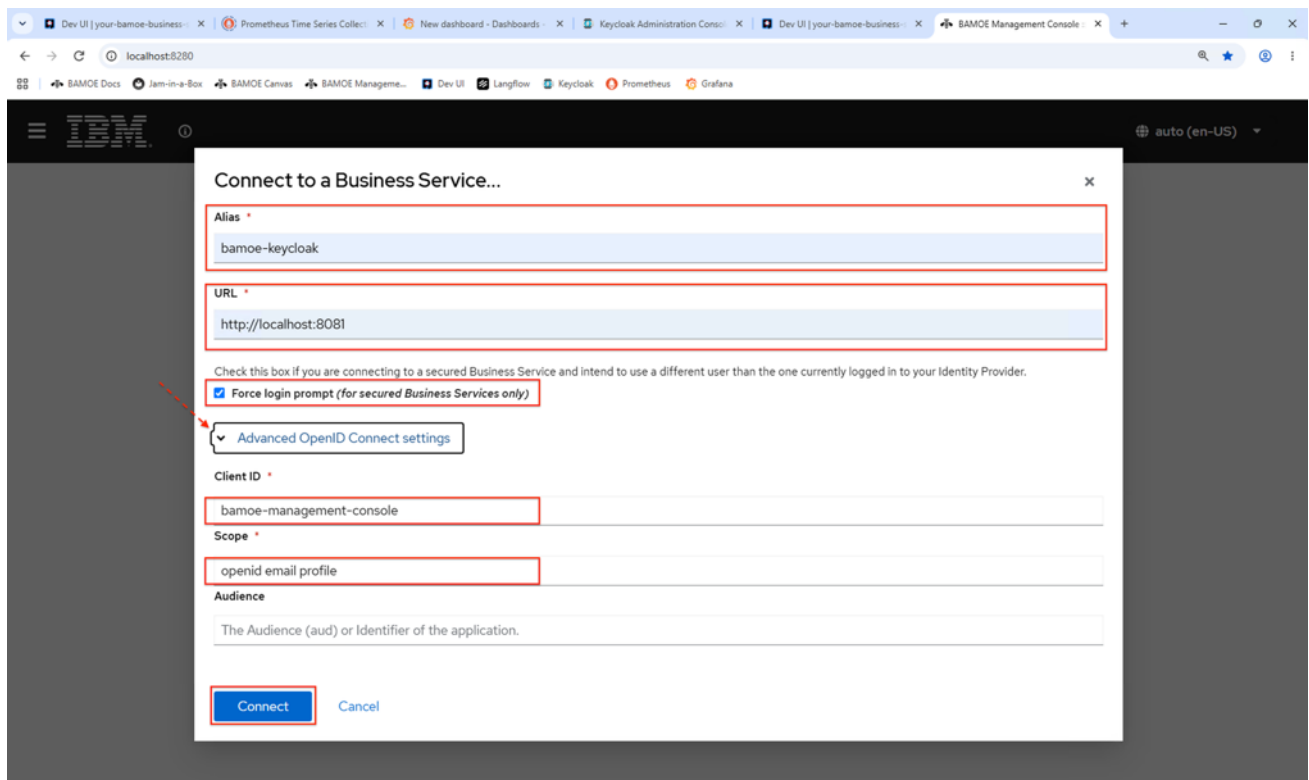
```
mvn quarkus:dev -Dquarkus.http.port=8081"
```

We will bring up the Business Service on port 8081, since port 8080 is already pre-configured in the Management Console.

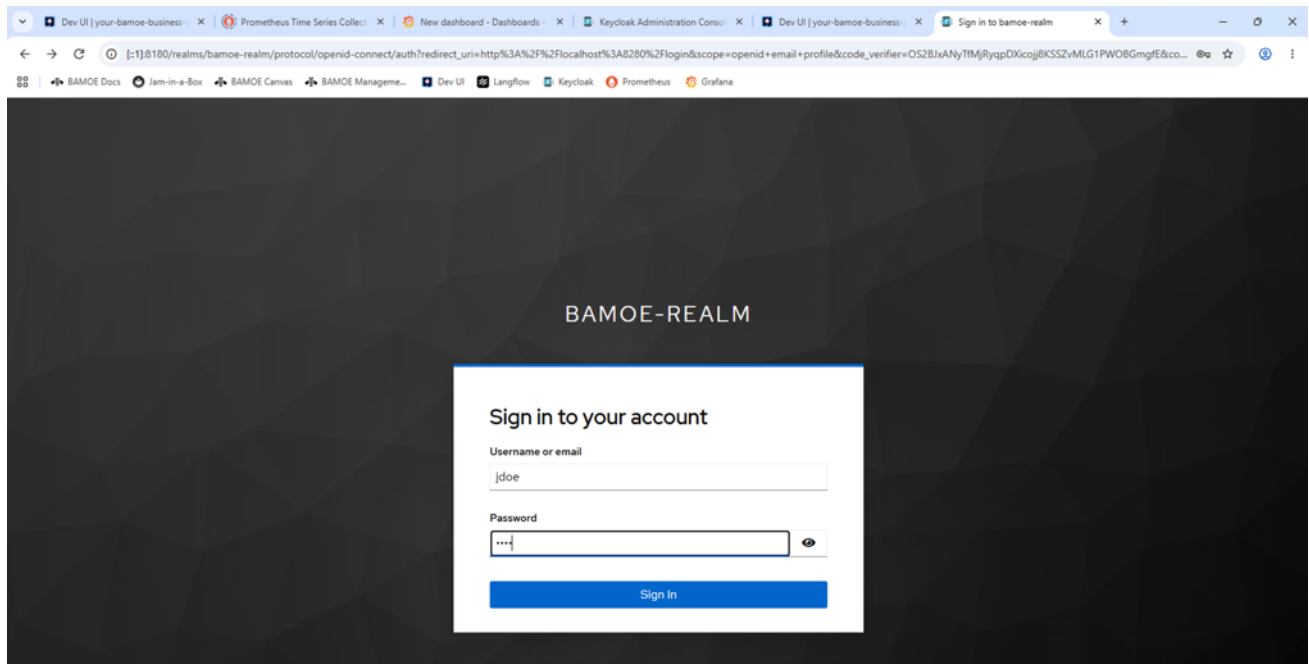
- Once the Business Service is running, open the Management Console in your Chrome browser and use: <http://localhost:8280>
- Note that port 8080 comes pre-configured, and you cannot change or remove it to include Keycloak authentication settings. Therefore, click **"Add"** to configure the application on port 8081.



- Enter the following settings:
 - Alias: bamoe-keycloak
 - URL: <http://localhost:8081>
 - Force login prompt: Turn ON
 - Client ID: bamoe-management-console
 - Scope: openid email profile
- Click **Connect**:



- Once you configure the Management Console, an authentication request via Keycloak will be triggered.
- Simply provide the username and password for "**idoe**". Click "**Sign In**"



BAMOE-REALM

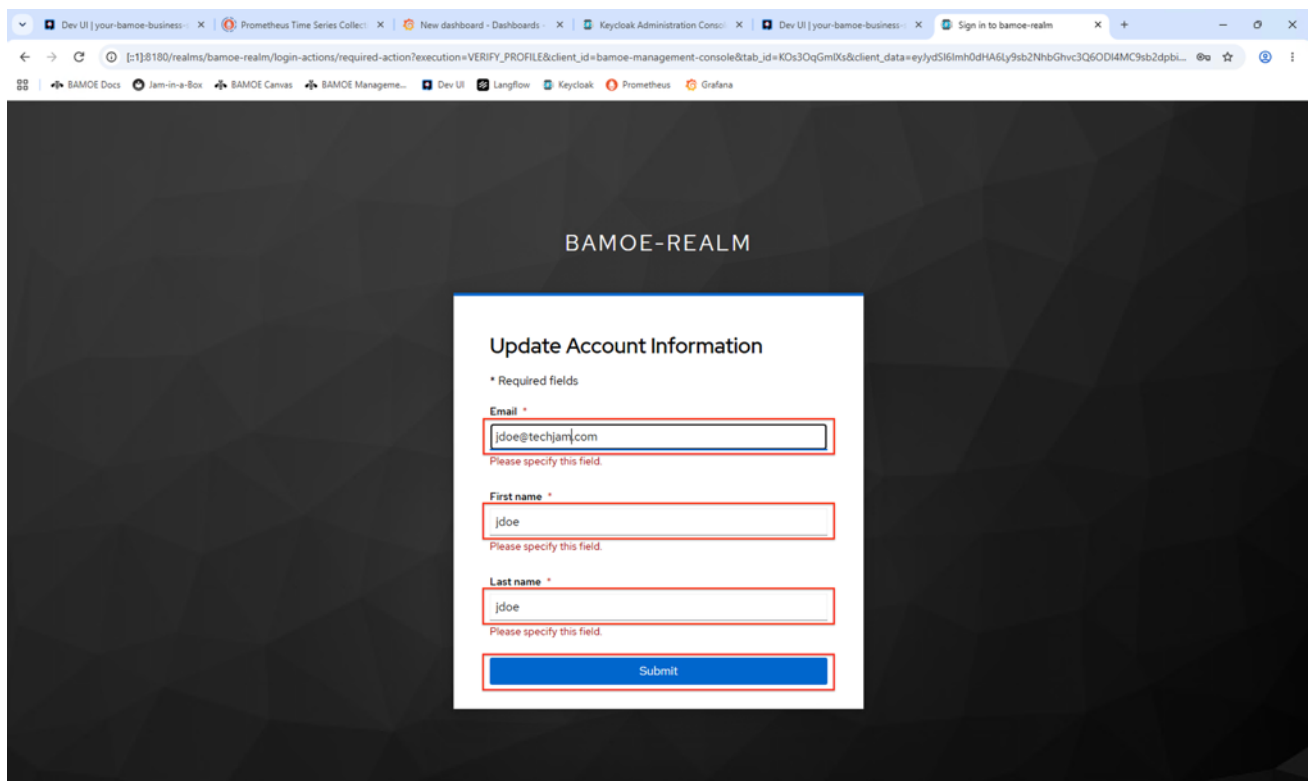
Sign in to your account

Username or email
jdoe

Password

Sign In

- It will be necessary to modify the data, please include fictitious data.



BAMOE-REALM

Update Account Information

* Required fields

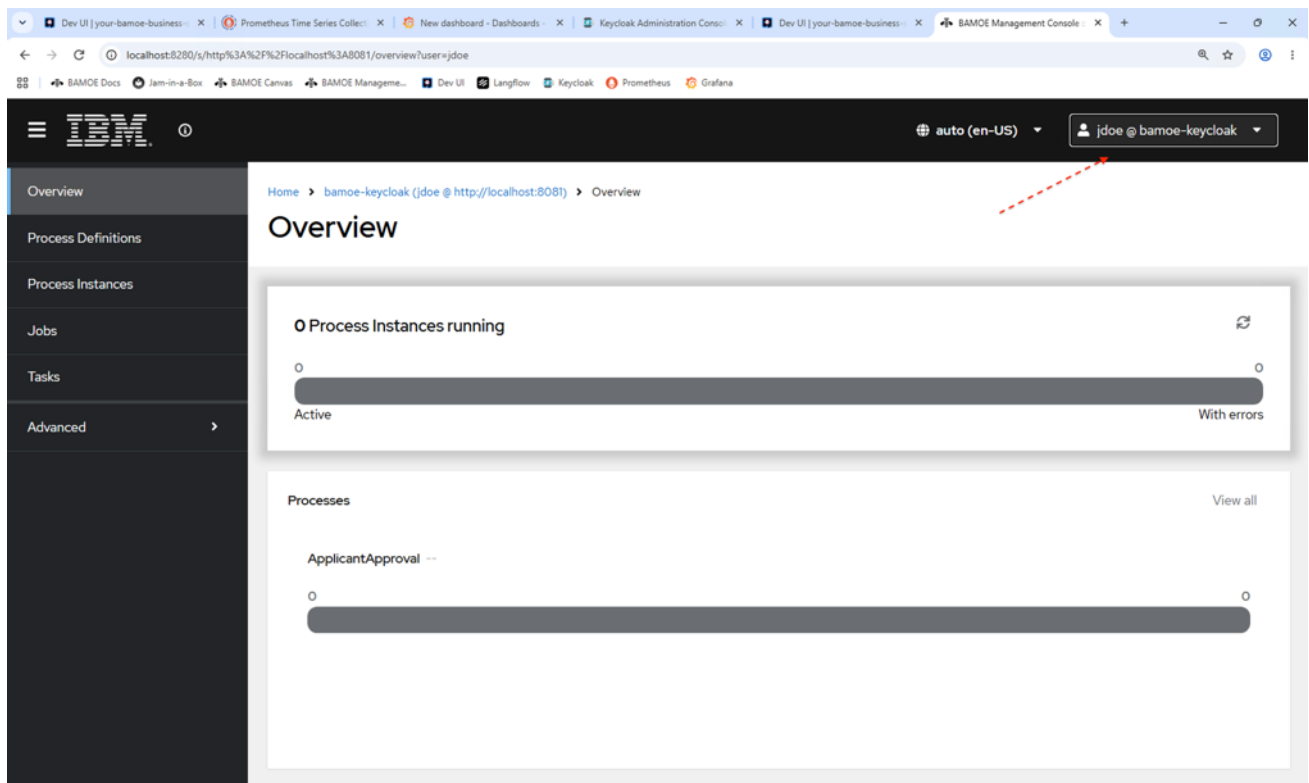
Email *
jdoe@techjam.com
Please specify this field.

First name *
jdoe
Please specify this field.

Last name *
jdoe
Please specify this field.

Submit

- Well done, authenticated user.



Explore mais sobre Keycloak pela página oficial <https://www.keycloak.org/>.

Congratulations! You have completed this lab.

Stay tuned for new product updates and official IBM documentation. I hope that from now on you will be able to create and explore your own use cases.

4 Consult Documentation and Communities

- [IBM BAMOE Official Documentation](#)
- [IBM Business Automation Community: Open Editions](#)